



SISTEMAS BIOINSPIRADOS

*MAGÍSTER EN INTELIGENCIA
ARTIFICIAL
AGOSTO 2026*

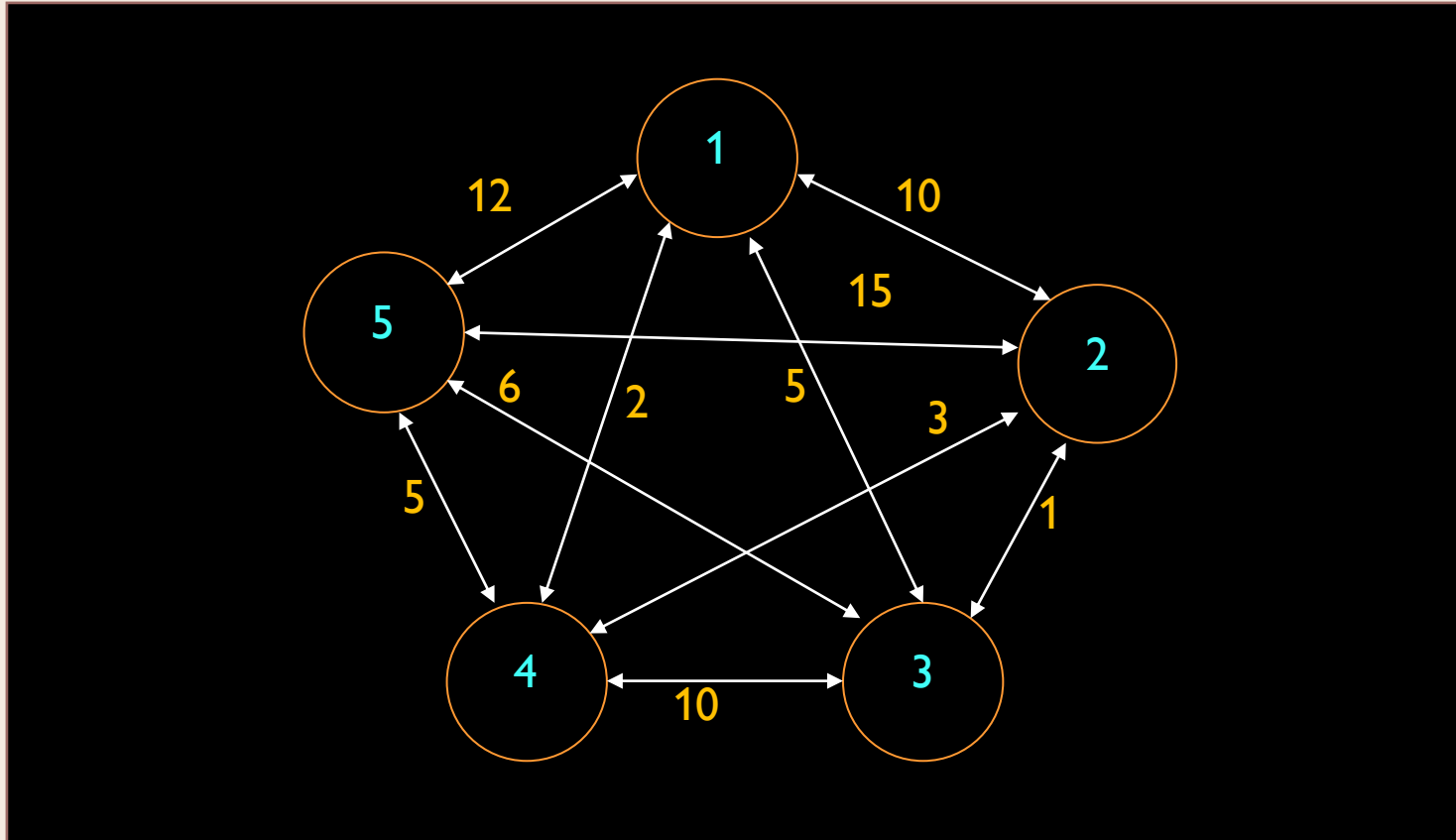
PROFESOR: RICARDO CONTRERAS A.



RETOMEMOS EL PROBLEMA DEL VENDEDOR VIAJERO (TSP)

- El vendedor viajero debe visitar cada ciudad exactamente una vez y luego regresar a su punto de partida.
- Dado el costo de viaje entre cada una de las ciudades, él debería planificar su itinerario de manera que el costo total sea mínimo.
- Este es un problema de optimización combinatoria que aparece en numerosas aplicaciones.

ESTUVIMOS MIRANDO ESTE EJEMPLO



REPRESENTACIÓN

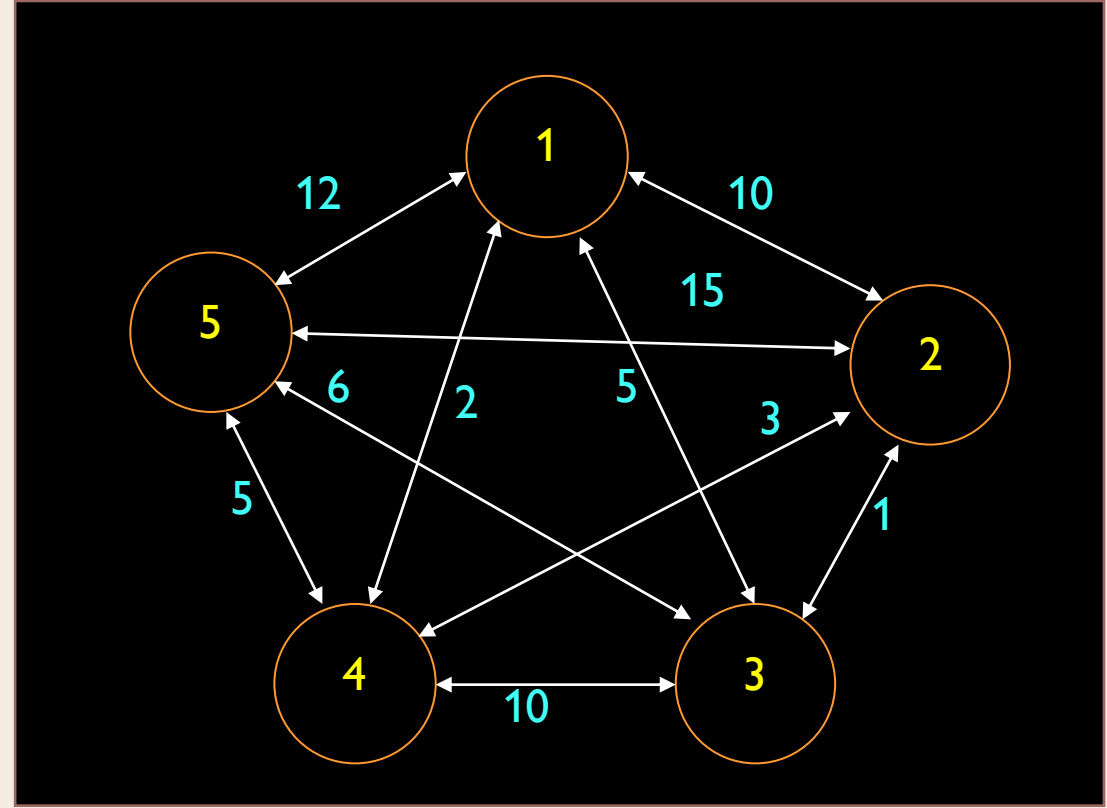
- La representación más natural es el vector de enteros
- $v = (i_1 i_2 \dots i_m)$ representa un itinerario: desde i_1 a i_2 , etc, desde i_{m-1} a i_m y regresa a i_1
- v es una permutación de $(1 \ 2 \ \dots \ m)$
 - (y hay que agregar la distancia al punto de inicio)
- ejemplos:
 - $v_1 = (4 \ 1 \ 5 \ 2 \ 3)$
 - $v_2 = (1 \ 3 \ 5 \ 4 \ 2)$
 - $v_3 = (4 \ 5 \ 2 \ 1 \ 3)$
 - $v_4 = (3 \ 5 \ 4 \ 1 \ 2)$

POBLACIÓN INICIAL

- Para el proceso de inicialización se puede usar alguna heurística
 - partiendo de diferentes ciudades de alta conectividad, por ejemplo
- Se puede inicializar la población creando un algoritmo que genere aleatoriamente permutaciones de $(1\ 2\ \dots\ m)$

FUNCIÓN DE EVALUACIÓN

- La evaluación del cromosoma está dada por la suma de los costos de viajar entre las ciudades



- $v1 = (4 \ 1 \ 5 \ 2 \ 3)$

$$f(v1) = 2 + 12 + 15 + 1 + 10 = 40$$

- $v2 = (1 \ 3 \ 5 \ 4 \ 2)$

$$f(v2) = 5 + 6 + 5 + 3 + 10 = 29$$

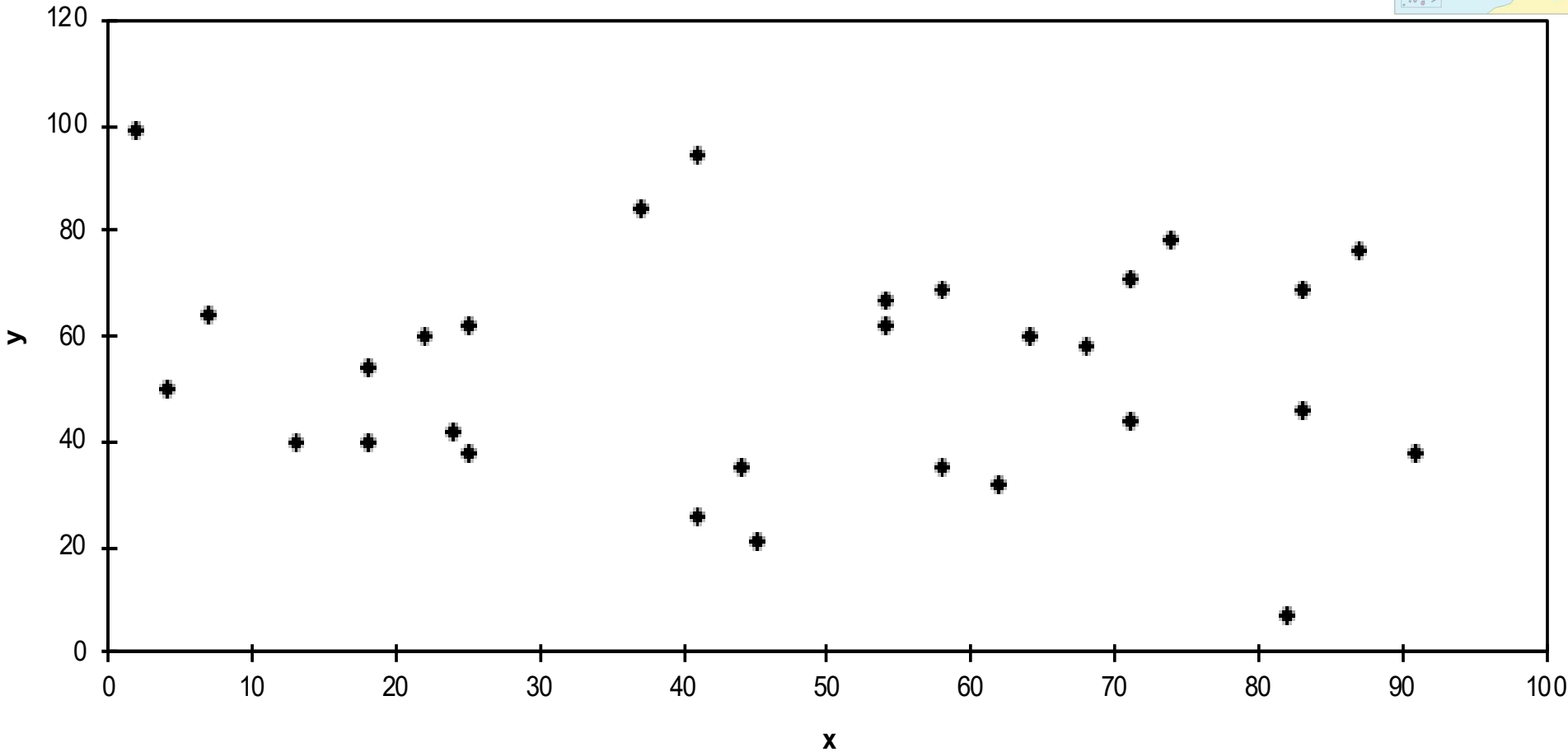
- $v3 = (4 \ 5 \ 2 \ 1 \ 3)$

$$f(v3) = 5 + 15 + 10 + 5 + 10 = 45$$

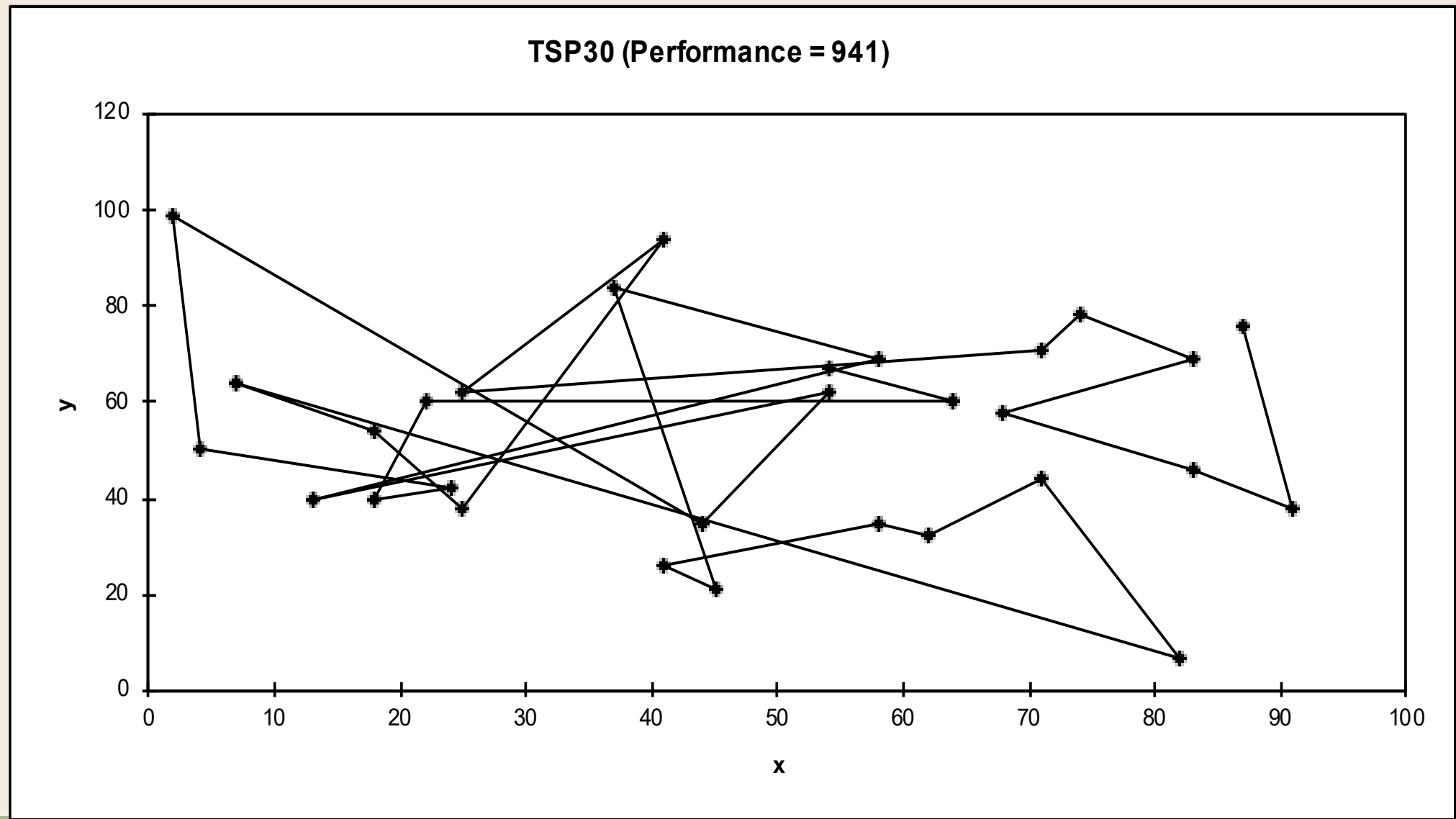
- $v4 = (3 \ 5 \ 4 \ 1 \ 2)$

$$f(v4) = 6 + 5 + 2 + 10 + 1 = 24$$

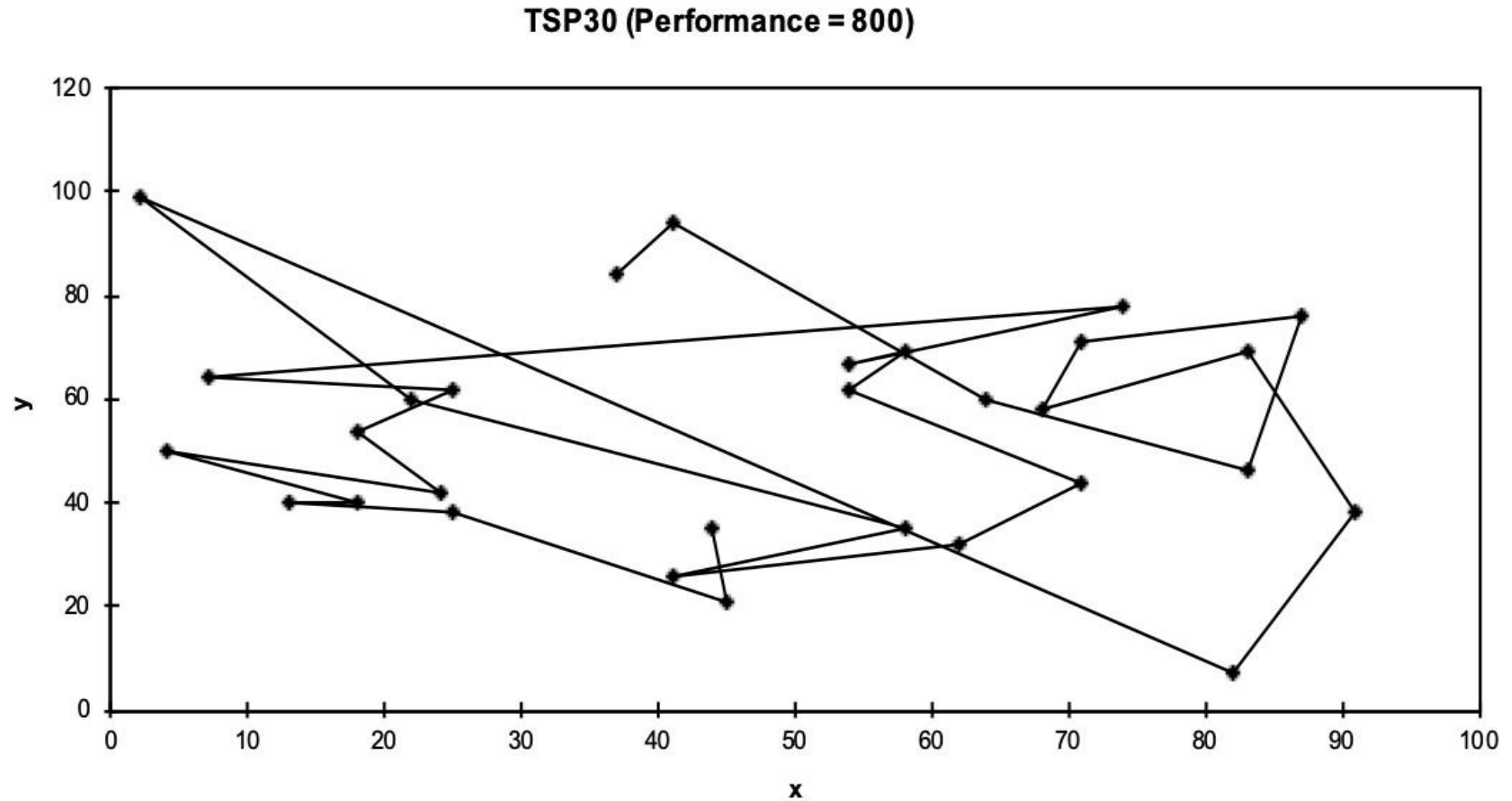
TSP EJEMPLO: 30 CIUDADES



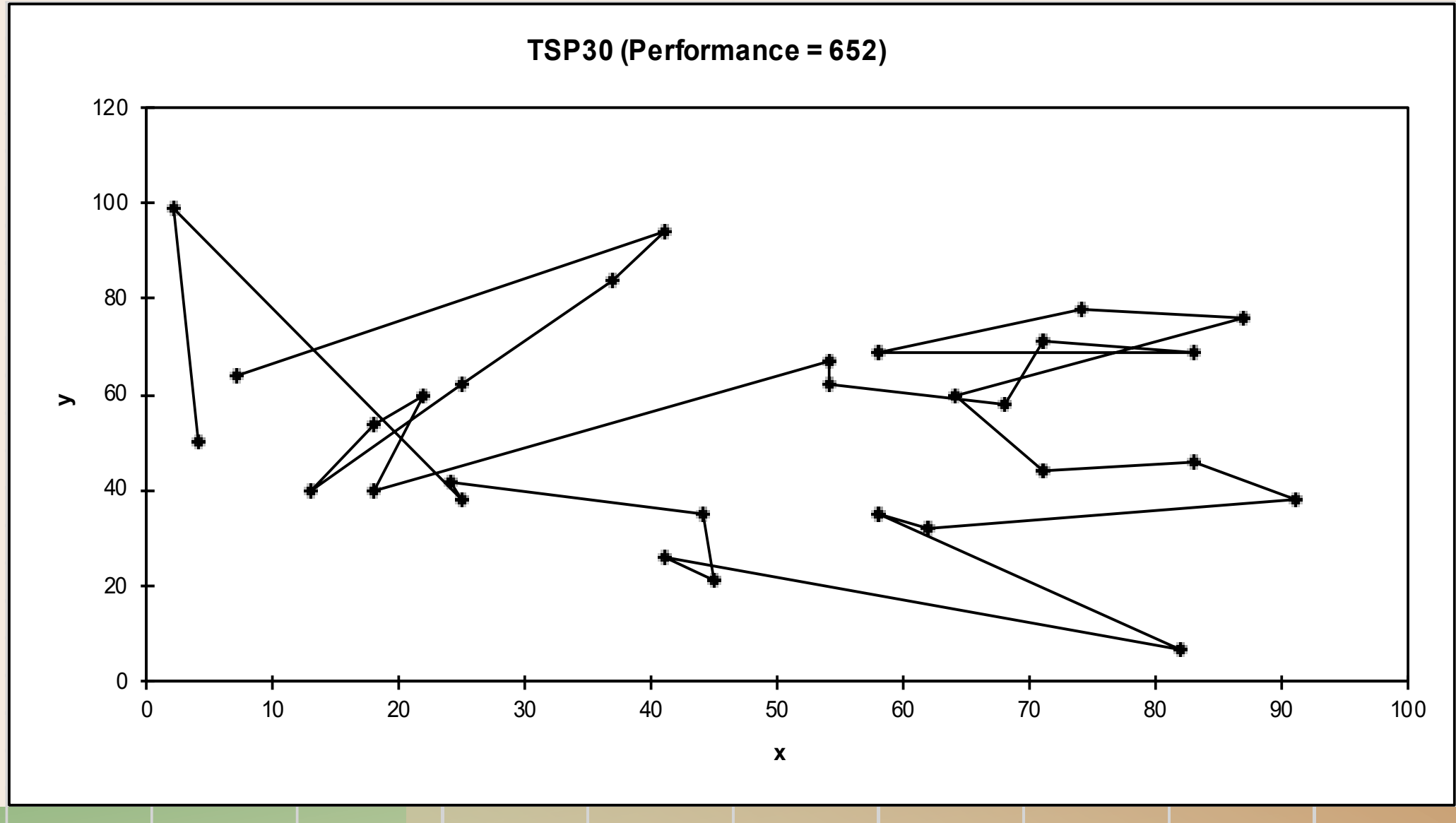
SOLUCIÓN, (DISTANCIA = 941)



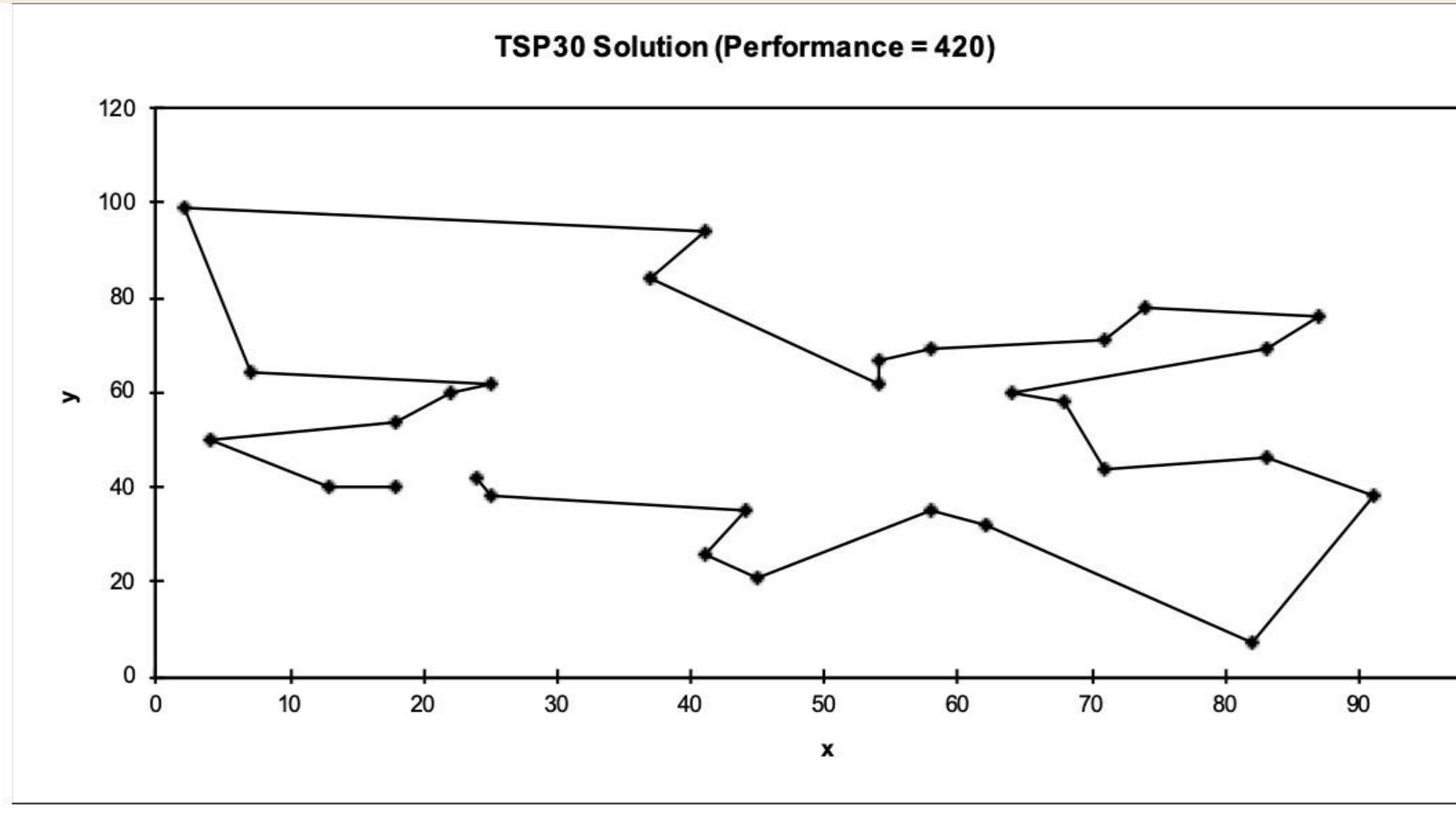
SOLUCIÓN_J (DISTANCIA = 800)



SOLUCIÓN_K (DISTANCIA = 652)

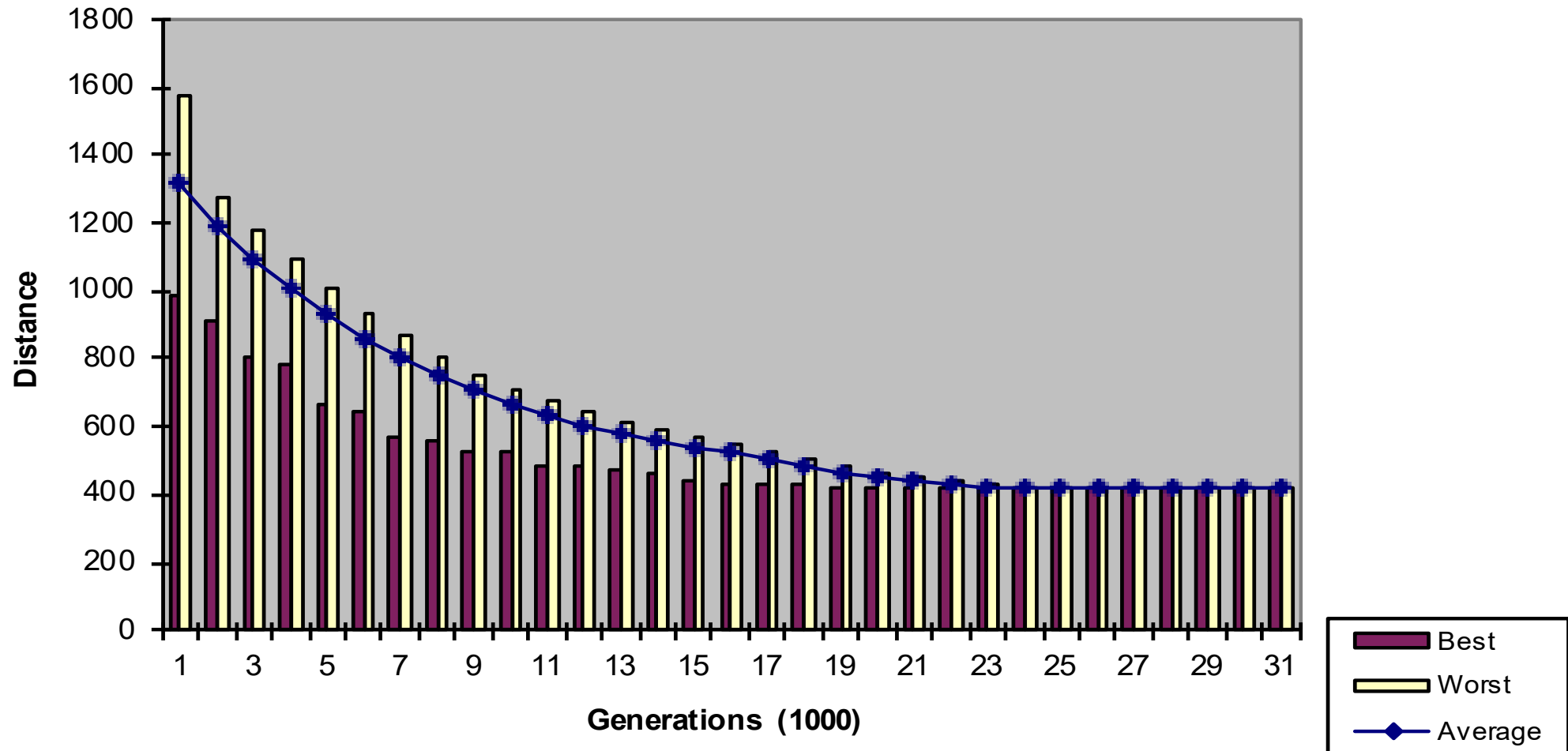


MEJOR SOLUCION (DISTANCIA = 420)



DE VERDAD VA MEJORANDO

TSP30 - Overview of Performance

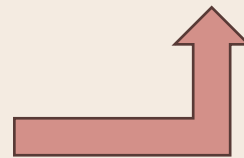


UN EJEMPLO SIMPLE PARA VER COMO LO VAMOS ENTENDIENDO

- Supongamos que tenemos una población que consta de los siguientes individuos:

Individuo	Valor	Fitness
1	111000	90
2	010101	240
3	001011	150
4	101110	320

Esta suma es importante

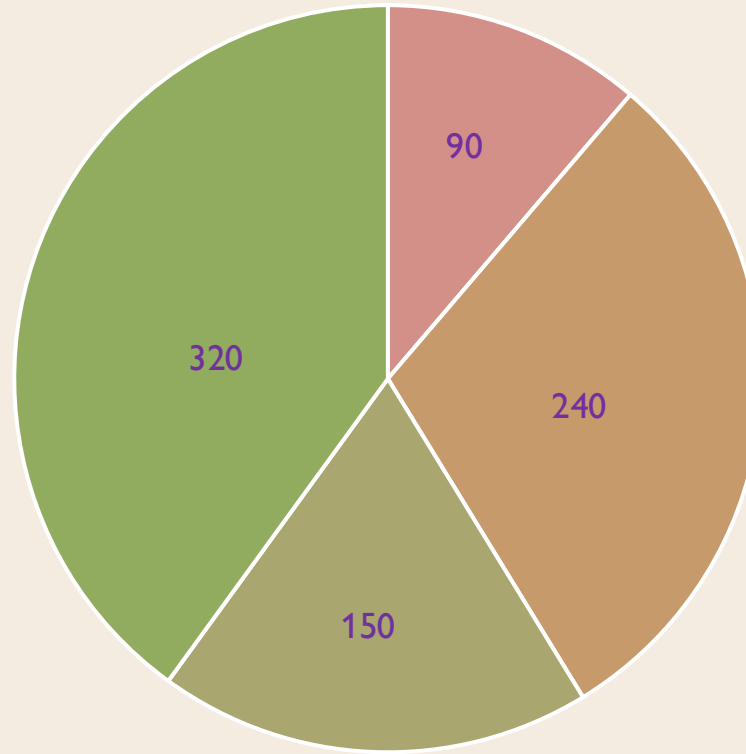


¿Por qué?

NUESTRA RULETA VA A TENER UNA FORMA EXTRAÑA...

1 - 90	91 - 330	331 - 480	481 - 800
---------------	-----------------	------------------	------------------

LA PODEMOS LLEVAR A UNA FORMA MÁS "AMIGABLE"



1 - 90	91 - 330	331 - 480	481 - 800
--------	----------	-----------	-----------

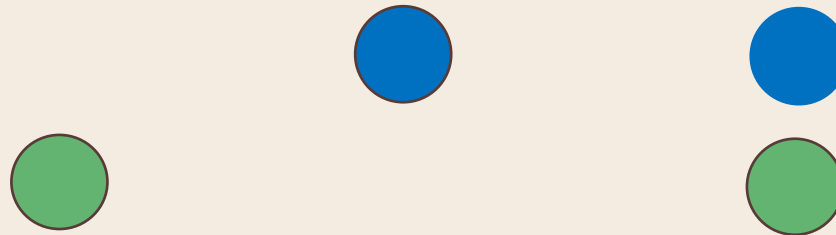
PARA USAR RULETA TENEMOS LOS SIGUIENTES NÚMEROS ALEATORIOS

- 450
 - 600
 - 300
 - 770
-
- ¿Cuáles serían los padres seleccionados?

TRABAJANDO CON LOS NÚMEROS ALEATORIOS Y LA RULETA

- 450
- 600
- 300
- 770

1 - 90	91 - 330	331 - 480	481 - 800
---------------	-----------------	------------------	------------------



PADRES

- Pareja 1: individuos 3 y 4

Individuo	Valor	Fitness
3	001011	150
4	101110	320

- Pareja 2: individuos 2 y 4

Individuo	Valor	Fitness
2	010101	240
4	101110	320

HIJOS: CRUZAMIENTO SIMPLE

- Para el cruzamiento, tenemos dos números aleatorios: 4 y 2
- ¿Cuáles serían los hijos resultantes?

Individuo	Valor	Fitness
3	001011	150
4	101110	320

HIJOS: CRUZAMIENTO SIMPLE

- Para el cruzamiento, tenemos dos números aleatorios: 4 y 2
- ¿Cuáles serían los hijos resultantes?

Individuo	Valor
3	001011
4	101110

- Hijos de la pareja 1:

Individuo	Valor
Hijo 1	001010
Hijo 2	101111

HIJOS: CRUZAMIENTO SIMPLE

- Para la pareja 2

Individuo	Valor
3	010101
4	101110



- Hijos de la pareja 2:

Individuo	Valor
Hijo 1	011110
Hijo 2	100101

VEAMOS CÓDIGO

- Primero veremos cómo generar población inicial
 - Vamos a generar diez cromosomas con ceros y unos, de largo 6

EXPLICACIÓN

- En este programa, la función `crear_poblacion` toma dos parámetros: el tamaño de la población y la longitud de cada individuo.
- Luego, genera una población de individuos binarios aleatorios y devuelve la lista que representa la población.
- La función `main` utiliza esta función para crear una población de diez individuos con vectores binarios de longitud 6 e imprime la población inicial.

CÓDIGO (CREAR POBLACIÓN (1))

```
import random
def crear_vectores(num_vectores, longitud):
    """
    Crea una lista de vectores binarios de longitud dada.

    Parámetros:
    - num_vectores (int): Número de vectores a crear.
    - longitud (int): Longitud de cada vector binario.

    Retorna:
    - list: Lista de vectores binarios creados aleatoriamente.
    """
    vectores = []
    for _ in range(num_vectores):
        # Generar un vector binario aleatorio de longitud dada
        vector = [random.choice([0, 1]) for _ in range(longitud)]
        vectores.append(vector)
    return vectores
```

CÓDIGO (CREAR POBLACIÓN (2))

```
def main():  
    # Parámetros para la creación de vectores  
    num_vectores = 10  
    longitud_vector = 6  
  
    # Crear los vectores aleatorios  
    vectores = crear_vectores(num_vectores, longitud_vector)  
  
    # Mostrar los vectores generados  
    print("Vectores generados:")  
    for i, vector in enumerate(vectores, 1):  
        print(f"Vector {i}: {vector}")  
  
    if __name__ == "__main__":  
        main()
```

RESULTADO DE LA EJECUCIÓN

Vectores generados:

Vector 1: [0, 1, 1, 0, 1, 1]

Vector 2: [0, 1, 0, 1, 0, 1]

Vector 3: [0, 0, 1, 0, 0, 1]

Vector 4: [1, 1, 1, 0, 1, 1]

Vector 5: [1, 1, 1, 0, 1, 0]

Vector 6: [0, 1, 1, 0, 1, 1]

Vector 7: [1, 0, 1, 0, 1, 1]

Vector 8: [1, 0, 0, 0, 0, 1]

Vector 9: [0, 0, 1, 1, 0, 0]

Vector 10: [1, 0, 0, 0, 1, 0]

CÓDIGO CRUZAMIENTO

- Ahora veremos cómo generamos hijos a través del cruzamiento
 - Lo haremos considerando solamente dos padres

EXPLICACIÓN

- En este programa, se define la función `cruzamiento_simple` que toma dos vectores binarios como entrada, selecciona un punto de corte aleatorio y realiza el cruzamiento simple para producir dos hijos.
- La función `main`
 - inicializa dos vectores binarios aleatorios,
 - muestra los padres,
 - aplica el operador de cruzamiento simple y
 - muestra los hijos resultantes.

CÓDIGO

```
import random
```

```
def cruzamiento_simple(padre1, padre2):
```

```
    """
```

Realiza el operador de cruzamiento simple entre dos vectores binarios.

Parámetros:

- padre1 (list): Primer vector binario de longitud 6.
- padre2 (list): Segundo vector binario de longitud 6.

Retorna:

- list: Dos nuevos vectores obtenidos mediante cruzamiento simple.

```
    """
```

CÓDIGO

```
# Seleccionar un punto de corte aleatorio
```

```
punto_corte = random.randint(1, len(padre1) - 1)
```

```
# Realizar el cruzamiento simple
```

```
hijo1 = padre1[:punto_corte] + padre2[punto_corte:]
```

```
hijo2 = padre2[:punto_corte] + padre1[punto_corte:]
```

```
return hijo1, hijo2
```

CÓDIGO

```
def main():  
# Inicializar dos vectores binarios aleatorios de longitud 6  
  
vector1 = [random.choice([0, 1]) for _ in range(6)]  
vector2 = [random.choice([0, 1]) for _ in range(6)]  
  
print("Padre 1:", vector1)  
print("Padre 2:", vector2)
```


CÓDIGO

```
# Aplicar el operador de cruzamiento simple
```

```
hijo1, hijo2 = cruzamiento_simple(vector1, vector2)
```

```
print("\nDespués del cruzamiento:")
```

```
print("Hijo 1:", hijo1)
```

```
print("Hijo 2:", hijo2)
```

```
if __name__ == "__main__":
```

```
    main()
```

RESULTADO DE LA EJECUCIÓN

Padre 1: [0, 0, 1, 0, 1, 1]

Padre 2: [1, 0, 1, 1, 1, 0]

Después del cruzamiento:

Hijo 1: [0, 0, 1, 1, 1, 0]

Hijo 2: [1, 0, 1, 0, 1, 1]

SIN EMBARGO, HAY UN PROBLEMA

Padre 1: [A, B, C, D, E, F]

Padre 2: [F, B, A, E, C, D]

Después del cruzamiento:

Hijo 1: [A, B, C, E, C, D]

Hijo 2: [F, B, A, D, E, F]

OTROS OPERADORES GENÉTICOS

- Cruzamiento
 - PMX (partially mapped) Goldberg y Lingle
 - OX (order) Davis
 - CX (cycle crossover) Oliver
- OX construye los hijos eligiendo una subsecuencia de un padre y preservando el orden relativo de los genes del otro padre. Los genes son copiados en el mismo orden partiendo del segundo punto de corte, omitiendo los que ya aparecieron.

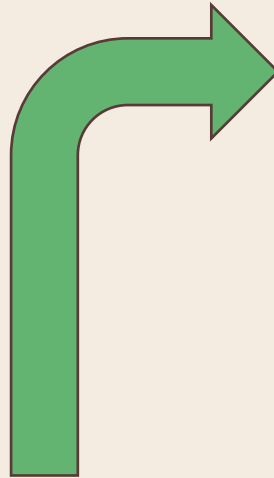
EL OPERADOR OX (VERSION 1)

- Padres:

- p1 = (7 3 1 8 2 4 6 5)
- p2 = (4 3 2 8 6 7 1 5)

- Elegir subsecuencia:

- p1 = (7 3 **1 8 2** 4 6 5)
- p2 = (4 3 **2 8 6** 7 1 5)



- Órdenes resultantes

- p1: 7 3 4 6 5
- p2: 4 3 7 1 5

- Hijos

- h1 = (6 5 | 1 8 2 | 7 3 4)
- h2 = (1 5 | 2 8 6 | 4 3 7)

EL OPERADOR OX (VERSIÓN 2)

El operador de **cruzamiento por orden (OX, Order Crossover)** en algoritmos genéticos combina dos cromosomas padres manteniendo el orden relativo de los elementos. Es ideal para problemas de permutación como el problema del vendedor viajero (TSP)

EL OPERADOR OX (VERSIÓN 2)

Datos iniciales

Padre 1 (P1): (1 2 3 | 4 5 6 | 7 8 9)

Padre 2 (P2): (4 5 2 | 1 8 7 | 6 3 9)

Puntos de corte: Posición 3 y posición 6 (marcados con barras).

PASOS PARA CONSTRUIR HIJO 1

- **Copiar la subcadena central de P1:** Se extrae el segmento entre los dos cortes del **Padre 1**: [4 5 6].
- Se coloca en la misma posición central del hijo: (_ _ _ | 4 5 6 | _ _ _).
- **Leer los elementos restantes de P2:** Se parte desde el segundo punto de corte (posición 6) en el **Padre 2** y se avanza de forma circular.
- El orden de los genes en P2 a partir de ahí es: 6, 3, 9, 4, 5, 2, 1, 8, 7.
- **Eliminar duplicados y rellenar:** Se omiten los números que ya están en la subcadena del hijo (4, 5 y 6).
- La lista depurada de P2 queda en este orden: 3, 9, 2, 1, 8, 7 (solo nos interesan los que faltan).
- **Completar los espacios vacíos del hijo:** Se rellenan los huecos del hijo en orden, empezando tras el segundo punto de corte (posición 6) y dando la vuelta al principio si es necesario:
 - Posiciones 7, 8 y 9: se colocan 3, 9 y 2.
 - Posiciones 1, 2 y 3: se colocan 1, 8 y 7.

Resultado Final

Hijo 1: (1 8 7 | 4 5 6 | 3 9 2)

EJEMPLO OX (BREVE)

- Padres:
 - $p1 = (1\ 2\ 3\ |\ 4\ 5\ 6\ 7\ |\ 8\ 9)$
 - $p2 = (4\ 5\ 2\ |\ 1\ 8\ 7\ 6\ |\ 9\ 3)$
- primer paso:
 - $h1 = (x\ x\ x\ |\ 4\ 5\ 6\ 7\ |\ x\ x)$
 - $h2 = (x\ x\ x\ |\ 1\ 8\ 7\ 6\ |\ x\ x)$
- Reordenando la secuencia en los padres:
 - $p2':\ 9\ 3\ 4\ 5\ 2\ 1\ 8\ 7\ 6$
 - $p1':\ 8\ 9\ 1\ 2\ 3\ 4\ 5\ 6\ 7$
- segundo paso:
 - $h1 = (2\ 1\ 8\ |\ 4\ 5\ 6\ 7\ |\ 9\ 3)$
 - $h2 = (3\ 4\ 5\ |\ 1\ 8\ 7\ 6\ |\ 9\ 2)$

ESTE MUNDO ES DE LOS VALIENTES

Solamente para estar seguros...

¿se atreven a generar el hijo 2?

HAY CASOS EN QUE LA MUTACIÓN TAMBIÉN DEBE SER CUIDADOSA

- Mutación
 - intercambio del contenido de 2 genes elegidos aleatoriamente
- Ejemplo
 - $p = (2 \ 1 \ 8 \ 4 \ 5 \ 6 \ 7 \ 9 \ 3)$
 - $h = (2 \ 1 \ 8 \ 4 \ 5 \ 3 \ 7 \ 9 \ 6)$

UN EJEMPLO DE MUTACIÓN (GENERAL) NO MUY EXITOSO

Población: 4 individuos

Longitud: 8

$P_{\text{cruz}} : 0.7$

$P_{\text{mut}} : 0.001$

Fitness: cantidad de 1s del string

Valores típicos de tamaño de población y longitud del genoma

Están en el rango 50-1000

Población inicial (generada al azar):

label	genoma	fitness
A	00000110	2
B	11101110	6
C	00100000	1
D	00110100	3

Después del cruzamiento...

label	genoma	fitness
E	1 0 1 1 0 1 0 0	4
	★	
B	1 1 1 0 1 1 1 0	6
	★	
C	0 0 1 0 0 0 0 0	1
F	0 1 1 0 1 1 1 0	5

Después de la mutación:

label	genoma	fitness
E'	1 0 1 1 0 0 0 0	3
B'	0 1 1 0 1 1 1 0	5
C	0 0 1 0 0 0 0 0	1
F	0 1 1 0 1 1 1 0	5

No nos está yendo muy bien

Antes

label	genoma	fitness
E	1 0 1 1 0 1 0 0	4
B	1 1 1 0 1 1 1 0	6
C	0 0 1 0 0 0 0 0	1
F	0 1 1 0 1 1 1 0	5

Después

label	genoma	fitness
E'	1 0 1 1 0 0 0 0	3
B'	0 1 1 0 1 1 1 0	5
C	0 0 1 0 0 0 0 0	1
F	0 1 1 0 1 1 1 0	5

- Para ello nos sirve el elitismo

- Si guardamos siempre el o los mejores, tenemos una estrategia (conservadora) para nunca correr el riesgo de disminuir el fitness.
- El criterio de selección del porcentaje de elitismo es más complicado de lo que parece.
- Depende de cuán conservadores seamos.



- MUCHOS PROGRAMAS PUEDEN FORMULARSE PARA UN PROBLEMA DADO. ESTOS PROGRAMAS PUEDEN SER DIFERENTES EN MUCHOS ASPECTOS

- Estructuras de datos utilizadas
- Operadores genéticos considerados
- Métodos para el manejo de restricciones
- Parámetros:
 - Tamaño de la población
 - Probabilidad de aplicación de operadores
 - etc.

ALGUNAS APLICACIONES

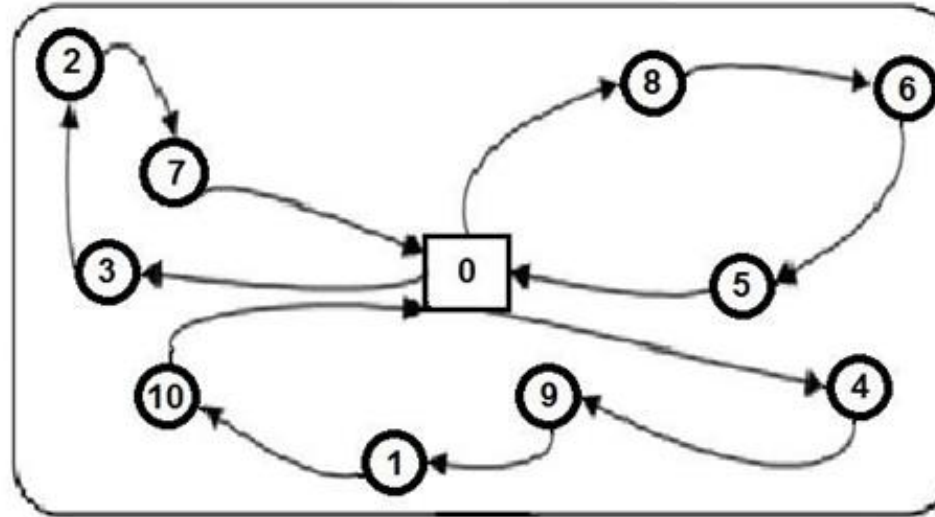
- VRP. Vehicle Routing Problem
- Bullying
- Formación de grupos
- Tratamiento de imágenes

VRP

- Es un problema de optimización combinatorial.
- Responde a la pregunta: ¿Cuál es el conjunto óptimo de rutas para una flota de vehículos que deben satisfacer los requerimientos de un conjunto de clientes?

EJEMPLO

Solución



Gene



Cromosoma

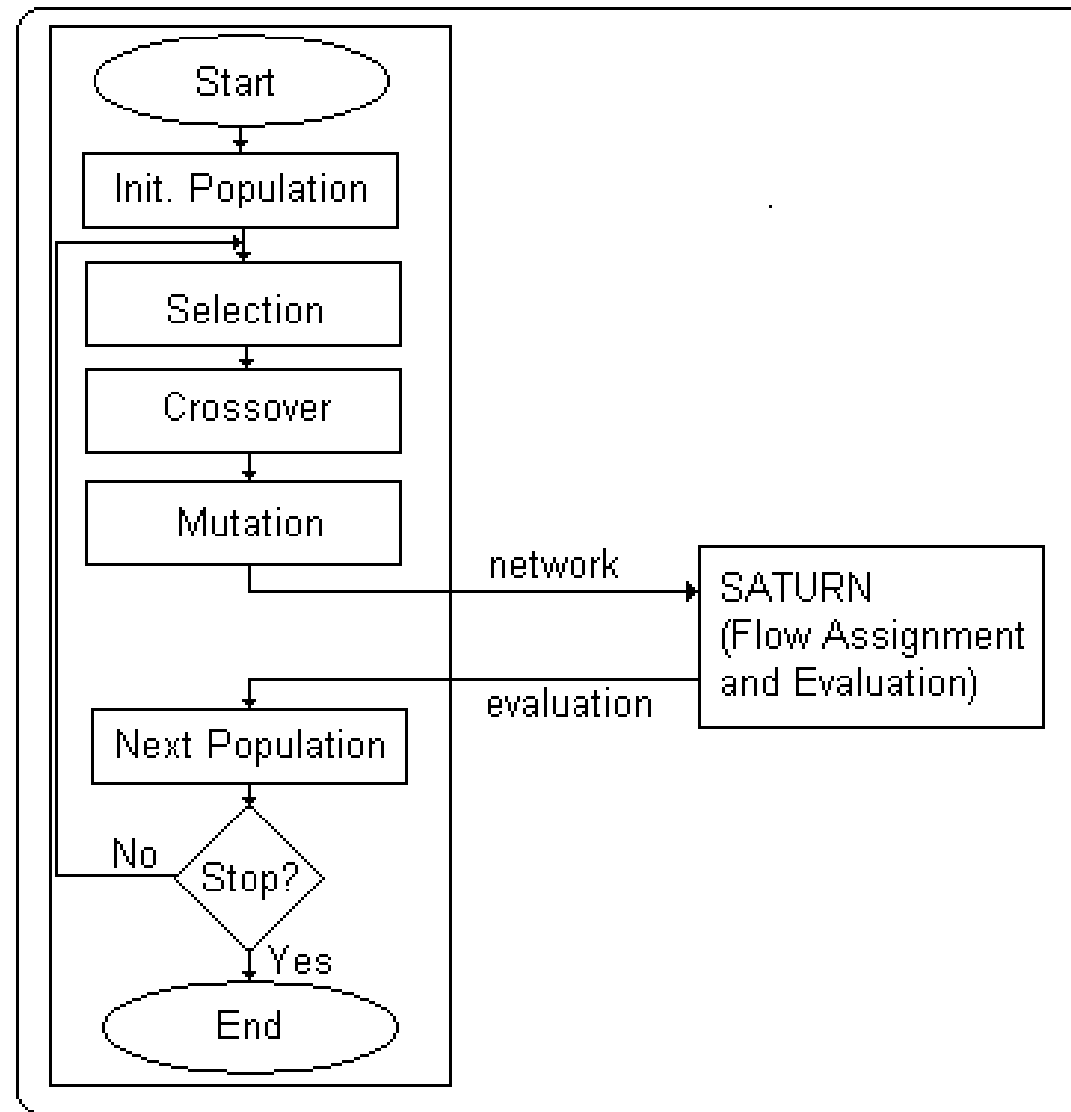


Significado

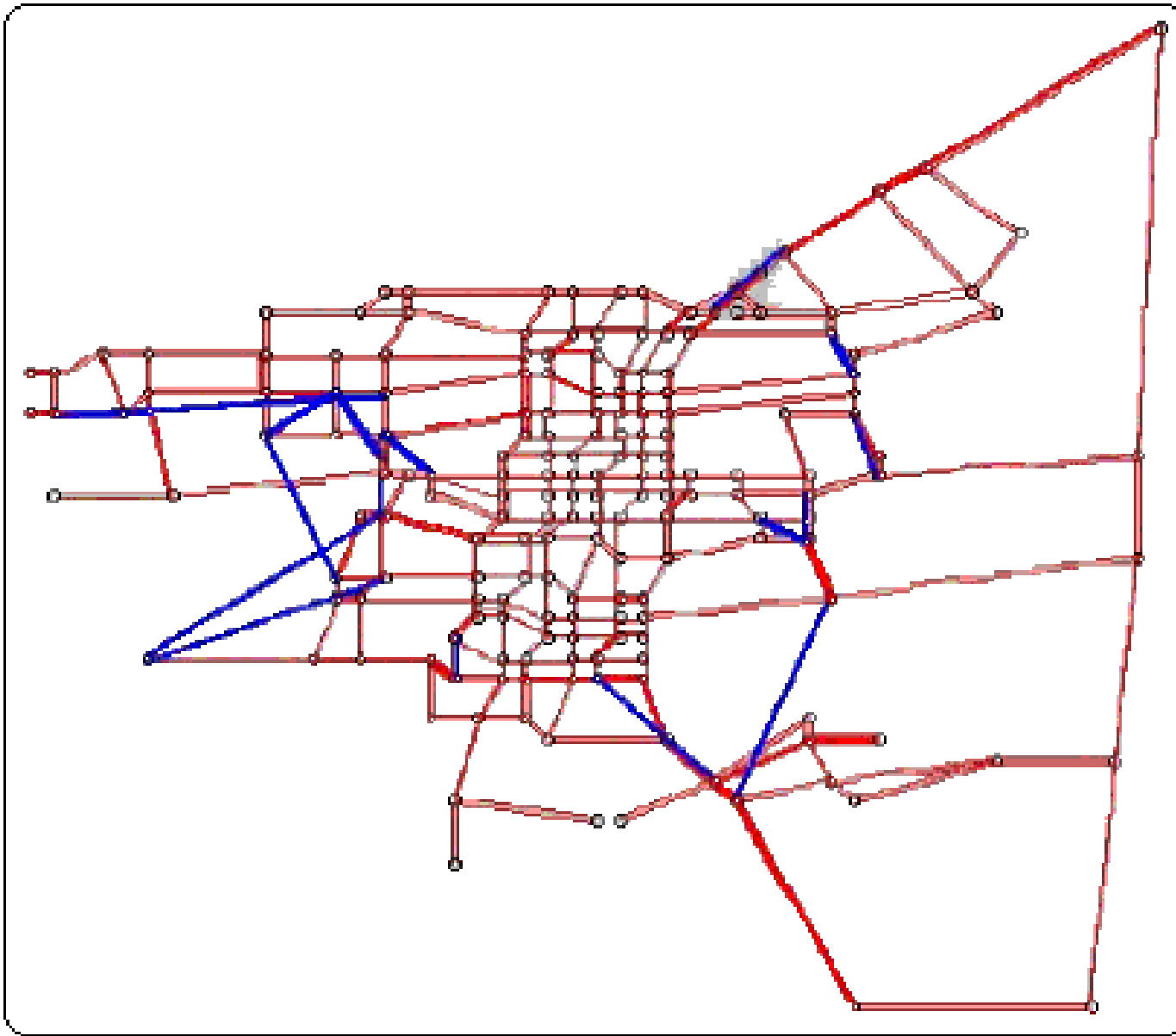
SANTA MARÍA DE LOS ANGELES



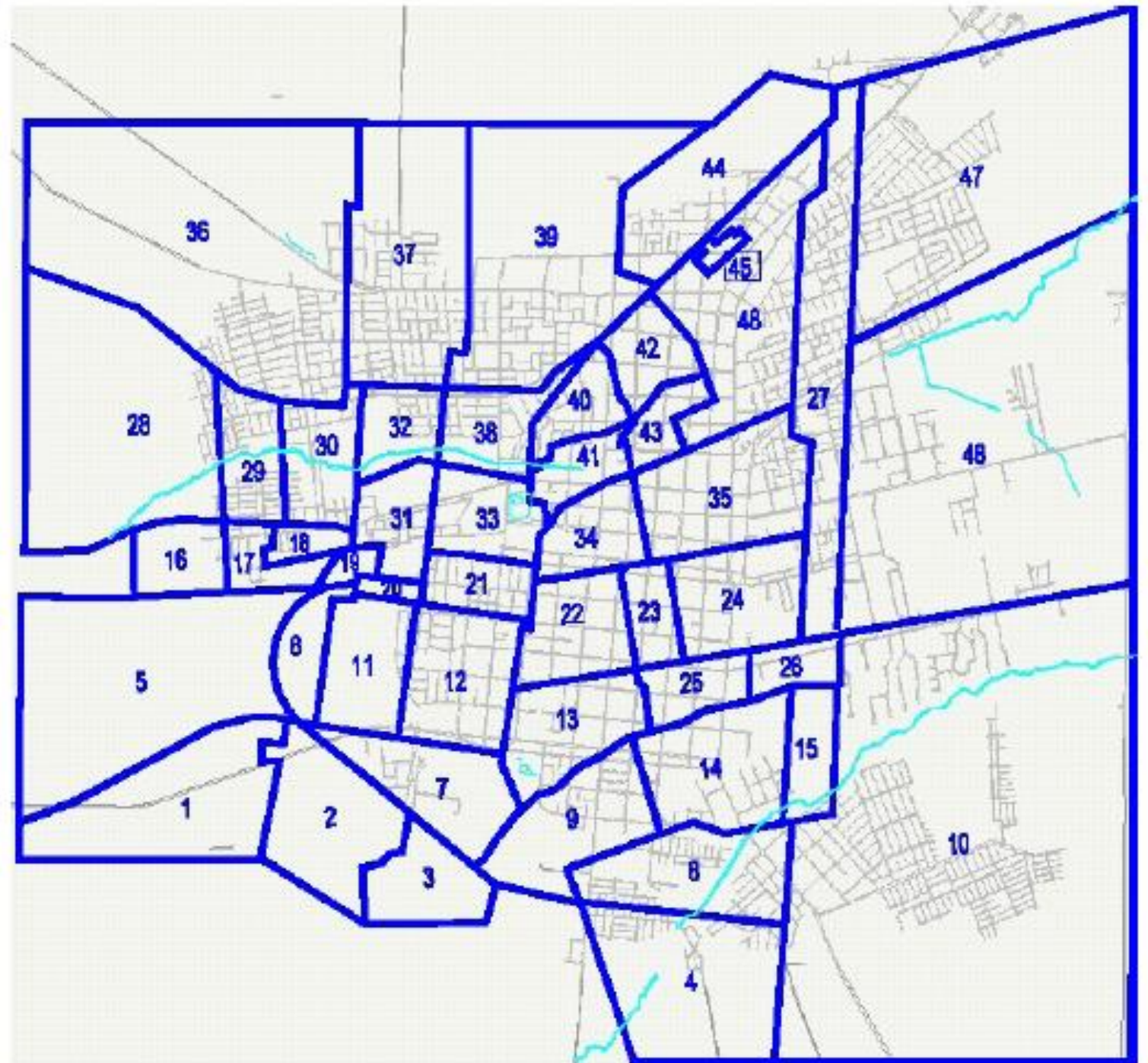
ESTRUCTURA ALGORITMO



EVOLUCIÓN DE CROMOSOMAS

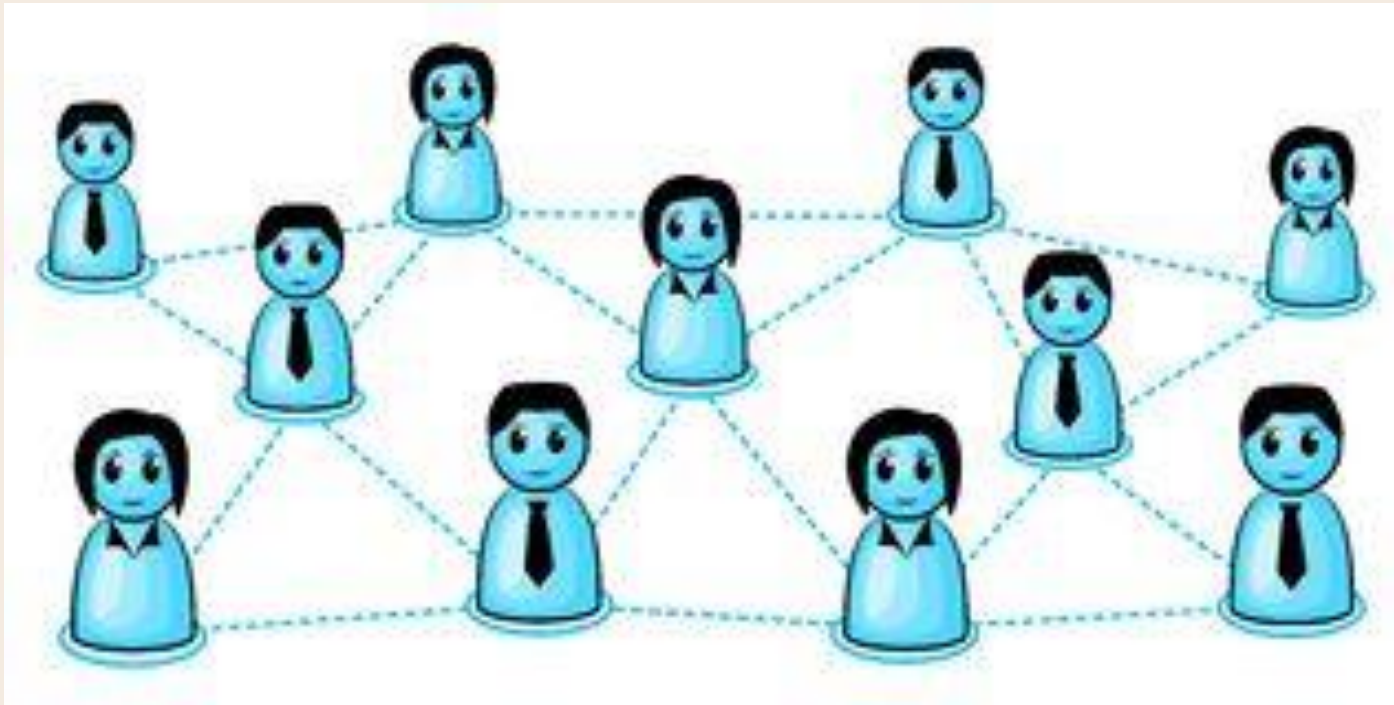


EN ESTE CASO
NO ERA
NECESARIO
CONSIDERAR
CADA
INTERSECCIÓN

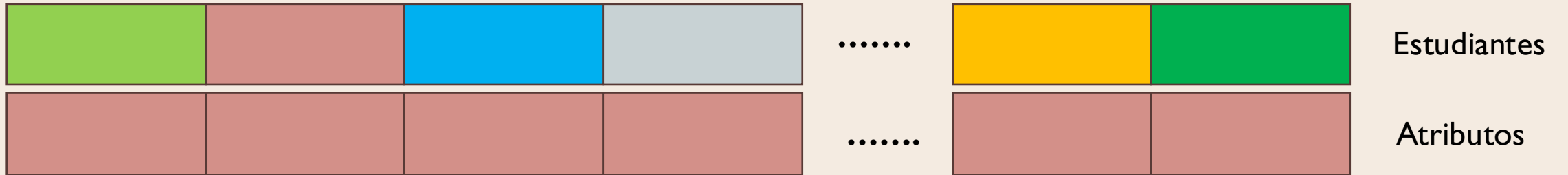


BULLYING

- ¿Cómo tratamos este problema?
- Sociograma: Una representación gráfica de las relaciones sociales entre individuos.

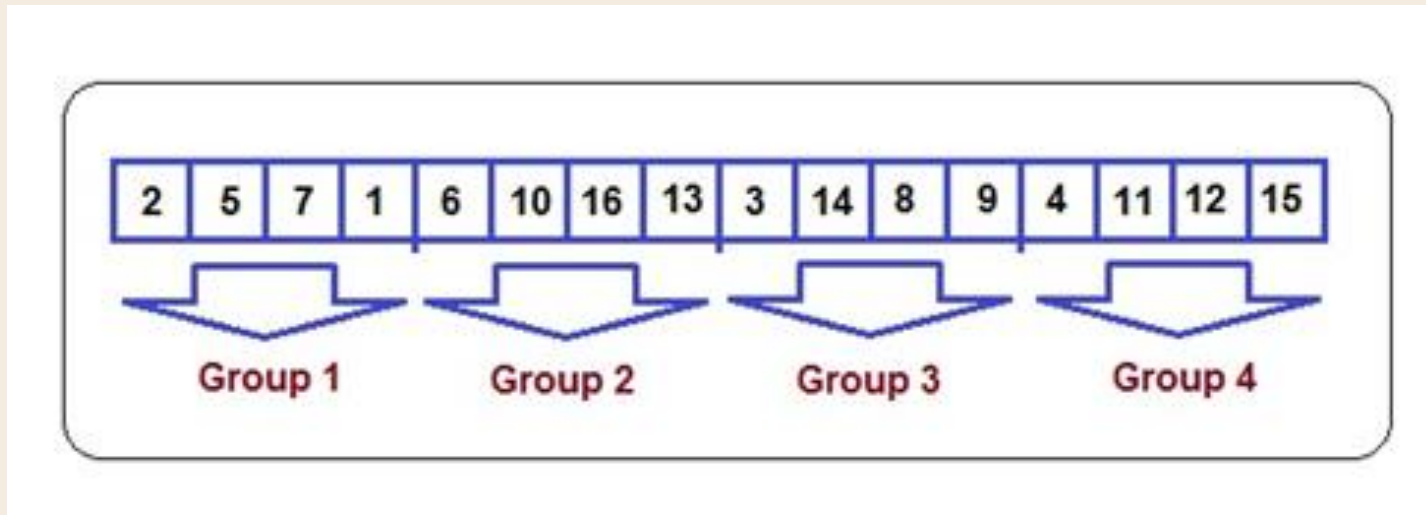


LOS CROMOSOMAS SE VEN ASÍ



FORMACION DE GRUPOS DE ESTUDIO

- Diferentes (y diversos) capacidades (tests).
- Diferentes estilos (tests).
- El desafío es fortalecer los grupos para un mejor desempeño.



Un individuo



CON NUMERITOS (LOS MEJORES)

Parámetros	Curso pequeño	Curso mediano	Curso grande
Población	200	300	900
Número de generaciones	50	460	260
Probabilidad de cruzamiento	100%	100%	70%
Probabilidad de mutación	4%	5%	4%

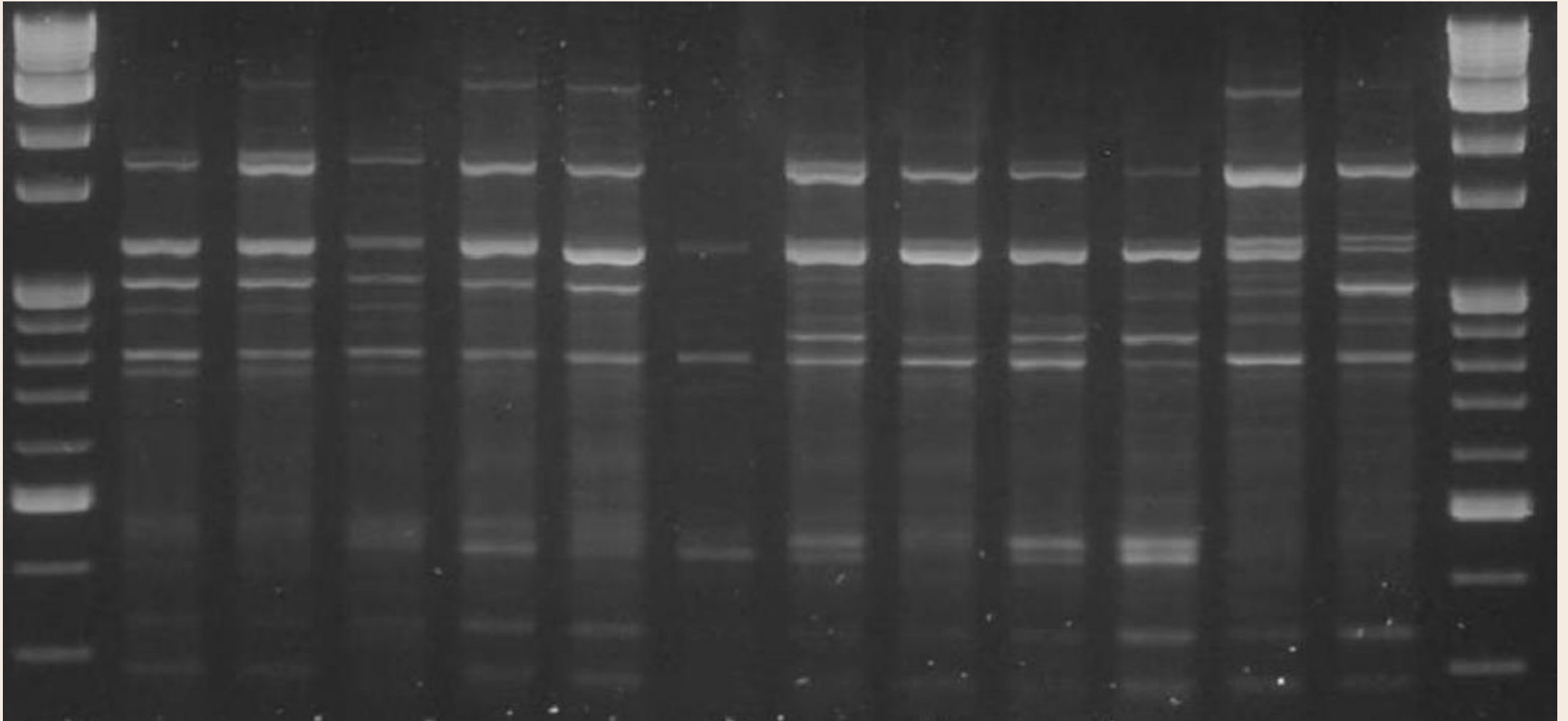
CORRECCIÓN DE IMÁGENES DE ADN

- Es hoy un problema relevante.
- Decisiones equivocadas pueden tener efectos catastróficos.
- La biotecnología está apoyada en herramientas que todavía no son precisas.

IMÁGENES RAPD

- RAPD: Randomly Amplified Polymorphism DNA
- Desarrolladores
 - J. Welsh y M. McClelland
 - J.G.K. Williams et al.
- Obtención de las imágenes
 - Gel
 - Campo eléctrico
 - Bromuro de etidio, luz ultravioleta
 - Cámara de alta resolución

EJEMPLO

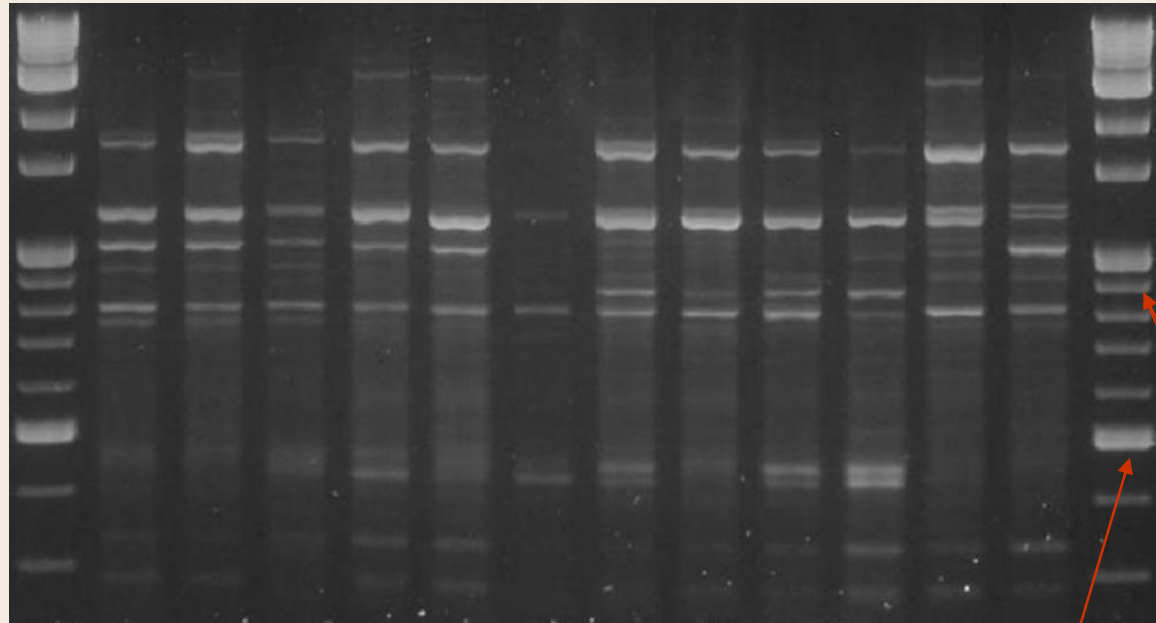


TRATAMIENTO TRADICIONAL

- Diversos paquetes software
- Intervención manual
- Costos y tiempos
- Errores

UNA IMAGEN RAPD TÍPICA

Eucalyptus globulus (muestras)



Carril = muestra

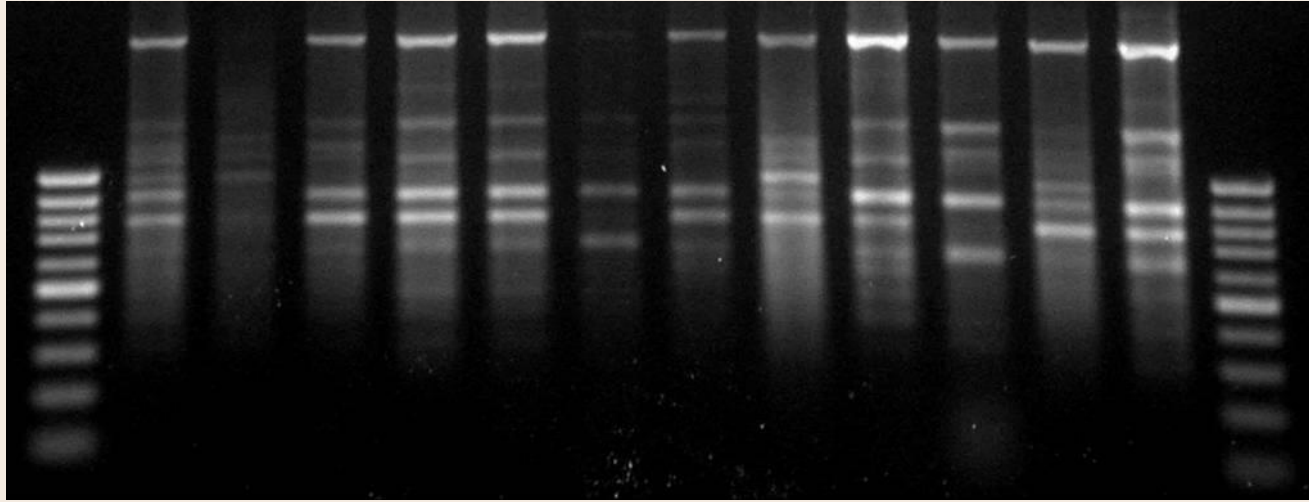
Banda = Peso molecular

PROBLEMAS

- En la generación de una imagen RAPD, muchos factores físico-químicos pueden impactar el proceso de electroforesis
 - Ruido
 - Rotaciones
 - Deformaciones
 - Otras distorsiones

ALGUNOS PROBLEMAS

Rotación de carriles



Banda con pendiente Banda curvada

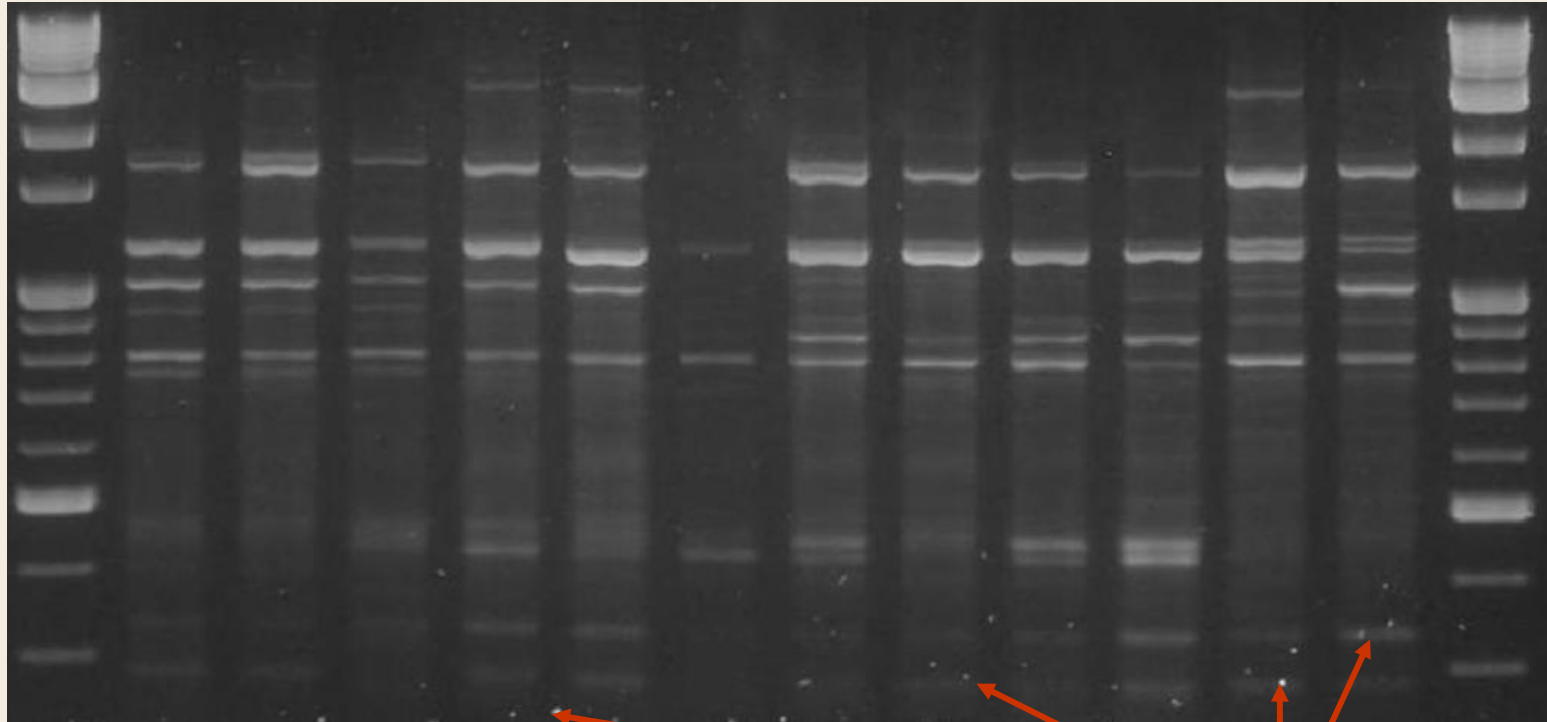
a)



b)



ENTRE OTROS PROBLEMAS

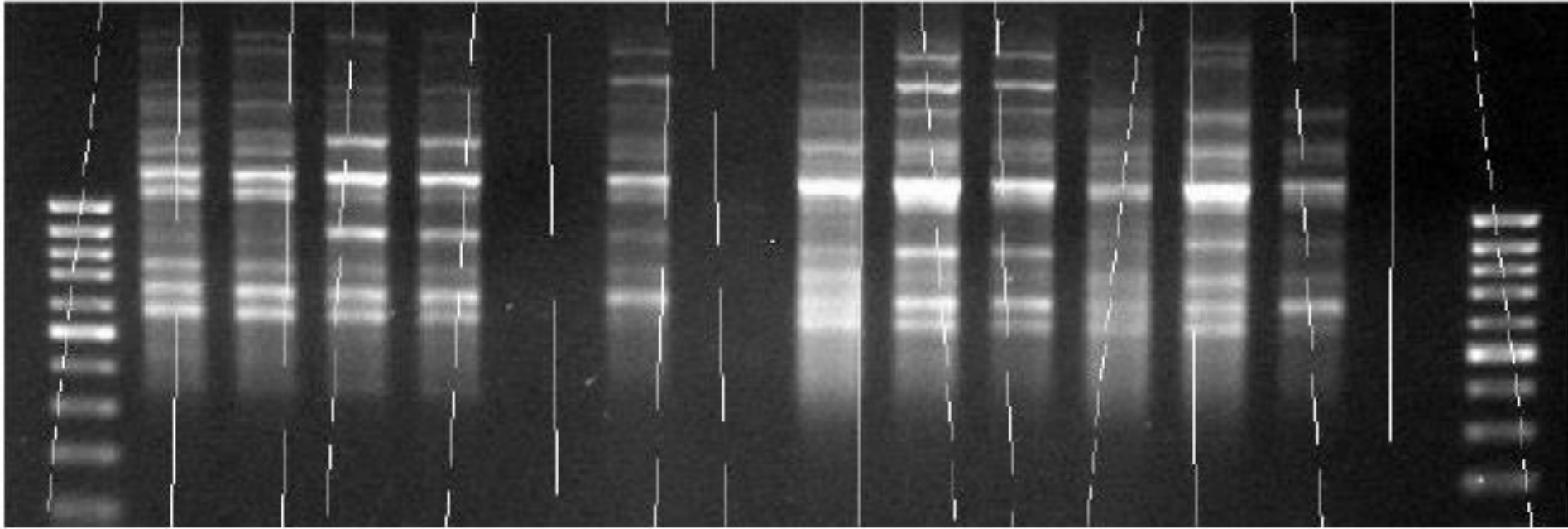


Polvo

PROPUESTA

- Algoritmos genéticos
 - Evolución biológica
 - Individuos (aptos)
 - Operadores
 - Función de evaluación

UNA MUESTRA DE *TEMPLATE*



- *AGs* permiten manejar un gran número de *templates*.
- Recordemos que son mecanismos de búsqueda global.
- Idea central: tener *templates* que sean similares a las imágenes originales.
- Es necesario encontrar una *función* que exprese esta similitud.
- Diferentes templates muestran diferentes *pendientes* para cada *carril*.

RESULTADOS (CORRECCIÓN CARRILES)

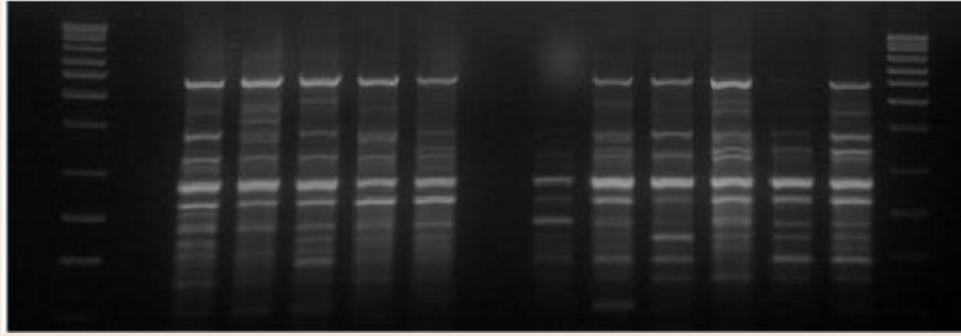
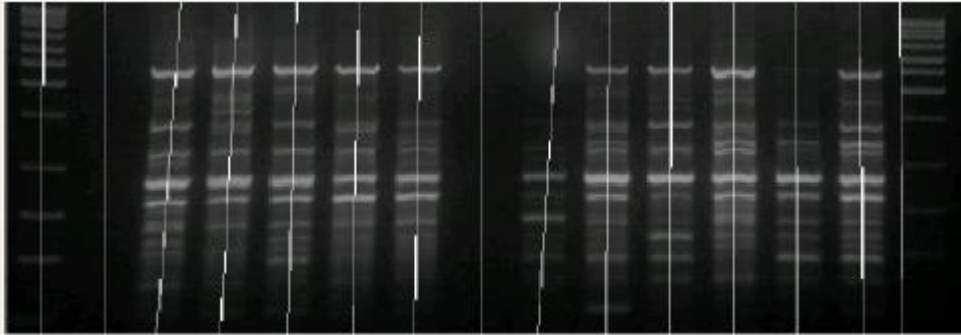


Imagen Original



Mejor template

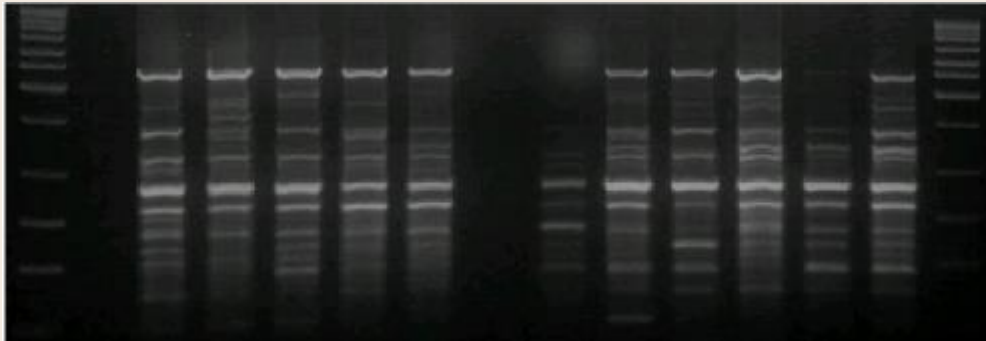


Imagen corregida

RESULTADOS EN CORRECCIÓN DE BANDAS

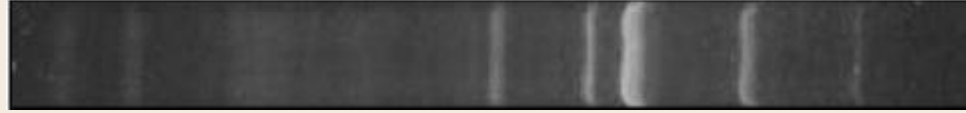


Imagen original

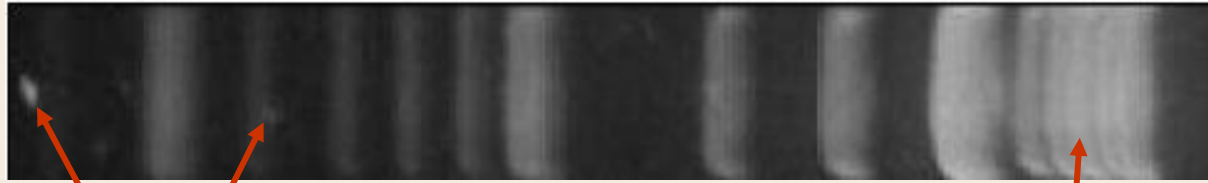


Mejor template



Imagen corregida

ALGUNOS PROBLEMAS PENDIENTES



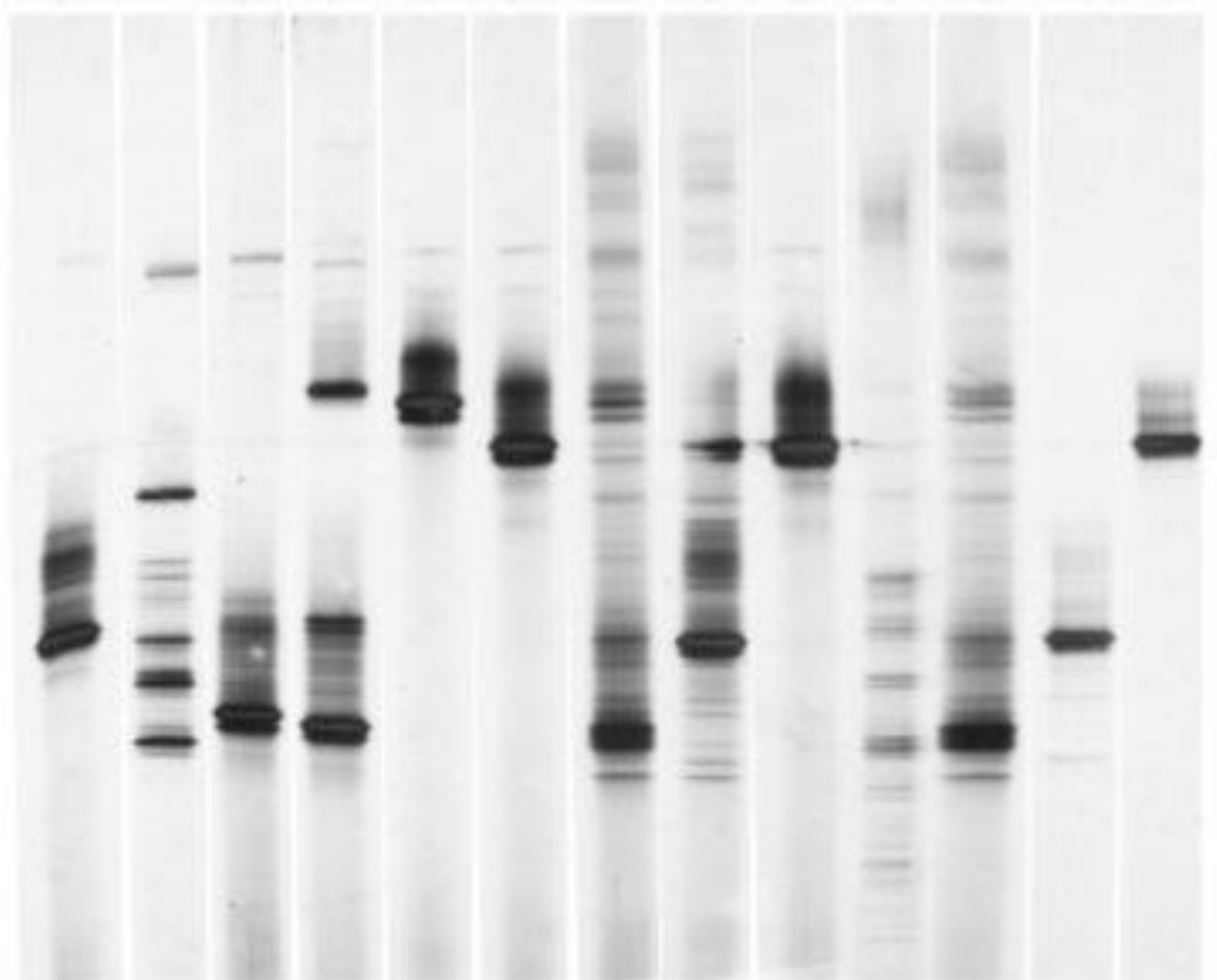
Polvo

Clusters

A VECES, SE PUEDE HIBRIDIZAR...

- Imágenes DGGE (Denaturing Gradient Gel Electrophoresis).
- Un tipo particular de imágenes obtenidas por electroforésis.
- Se usan para diferentes propósitos. En el caso que se menciona: estudio sobre biodiversidad microbiana.
- Hibridizado con Tabu Search y con Simulated Annealing.

IMÁGENES DGGE

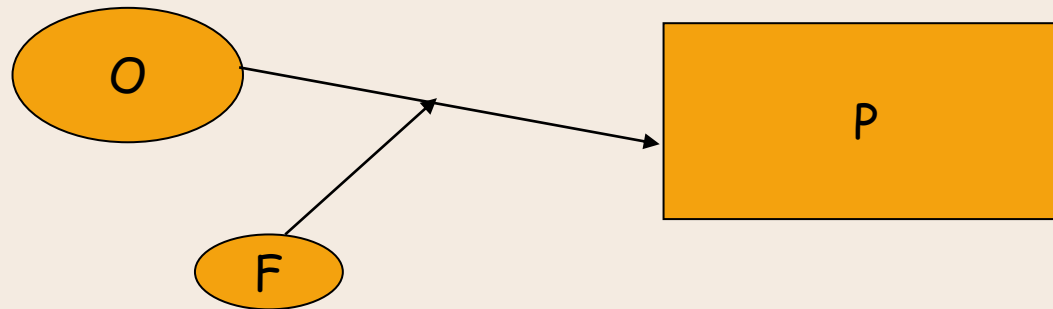


COMPUTACIÓN EVOLUTIVA

- ❖ Normalmente, la analogía es pobre entre:
 - ❖ Estructuras de datos y genotipos,
 - ❖ Soluciones y fenotipos
 - ❖ Operadores y mecanismos genéticos de transformación natural (mutación, recombinación, etc.)
 - ❖ *Fitness* usado de forma mediática en los procesos de selección.
- ❖ Más cercana a la replicación que a la selección *natural*
- ❖ Algoritmos genéticos, Estrategias evolutivas, Programación genética, Programación evolutiva ...

COMPUTACIÓN EVOLUTIVA

- ❖ Familia de métodos directos de búsqueda, estocásticos, basados en la población
- ❖ $P[t] = O[t] \times P[t-1]$
- ❖ P es una población de estructuras de datos que representan soluciones a un problema particular
- ❖ O es un conjunto de operadores de transformación usados para crear nuevas soluciones; F es una función de *fitness*



COMPUTACIÓN EVOLUTIVA

- ❖ ¿Es mágica? *No.*
- ❖ ¿Es universal? *No. Muy buena para ciertos problemas, muy mala para otros.*
- ❖ ¿Es fácil de aplicar? *A veces ...*
- ❖ ¿Por qué deberíamos interesarnos en CE? *A veces se obtienen mejores resultados que con técnicas más tradicionales (no siempre). Buen marco general dentro del cual es posible crear algunos métodos de amplio poder para problemas específicos*
- ❖ Usos: Optimización, problemas combinatoriales tales como Scheduling, Alife, Biología teórica, entre otros.

¿PARA QUÉ SIRVE?

❖ A veces, en combinación con otros métodos, es útil para:

- ❖ Optimización variables complejas
- ❖ Muchos problemas de optimización combinatorial
- ❖ Mezcla de problemas de optimización discretos-continuos
- ❖ Bases de evolución artificial
 - ❖ Diseño
 - ❖ Espacios de búsqueda de dimensiones desconocidas o variables

OPTIMIZACIÓN Y BÚSQUEDA

- ❖ Técnicas determinísticas clásicas (a menudo para problemas con variables continuas)
 - ❖ Métodos directos de búsqueda (sólo evaluaciones de funciones)
 - ❖ Métodos de gradiente descendente (hacen uso de información de gradiente, como en la regla delta)
- ❖ Operan en un espacio (completo o parcial) de soluciones posibles, saltando de una solución a la siguiente
 - ❖ Evaluativo
 - ❖ Heurístico
 - ❖ Estocástico

MÉTODOS DIRECTOS DE BÚSQUEDA

❖ ¿Cuándo se usan?

- ❖ La función a ser optimizada no es derivable, o está sujeta a errores random;
- ❖ Las derivadas de la función no son continuas, o su evaluación es muy cara y/o compleja.
- ❖ Falta de tiempo para aplicar métodos computacionales más costosos basados en gradiente.
- ❖ Puede que se requieran soluciones aproximadas en cualquier momento del proceso de optimización (los métodos directos trabajan refinando iterativamente la solución).

SIMULATED ANNEALING



- *Inspirado en el annealing de metales*

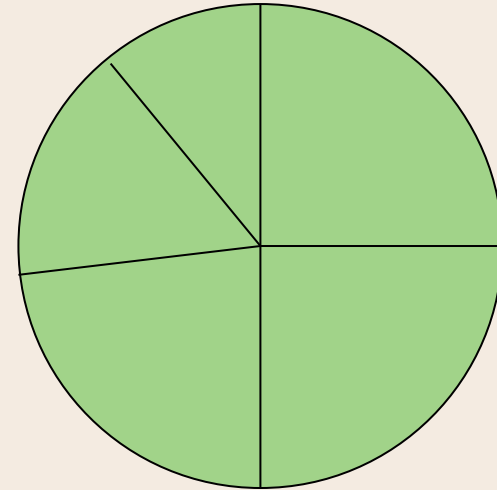
- 1) Inicialice T (análogo a la temperatura), genere una solución inicial, S_c , el costo de esta solución es C_c .
- 2) Use un operador para generar de forma random una nueva solución S_n a partir de S_c , con costo C_n .
- 3) Si $(C_n - C_c) < 0$, i.e. Se encontró solución mejor, hacer $S_c = S_n$. En caso contrario $\text{costo}[-(C_n - C_c)/T] > \text{random}$, entonces $S_c = S_n$, se acepta el movimiento con probabilidad proporcional a $\exp[-(C_n - C_c)/T]$.
- 4) Reduzca T , en general linealmente con el número de iteraciones.
- 5) Si se alcanza el criterio de detención, fin; en caso contrario ir a paso (2).

FORTALEZA DE LOS AES

- ❖ En algún sentido, AEs atacan los problemas desde una perspectiva global, en vez de usar perspectivas puramente locales.
- ❖ Ya que se basan en la población, si se tratan adecuadamente se pueden explorar múltiples áreas del espacio de búsqueda en paralelo.
- ❖ Los elementos estocásticos en los algoritmos implican que no están necesariamente forzados a encontrar el óptimo local más cercano (como ocurre con los algoritmos determinísticos de búsqueda local).

SELECCIÓN: RULETA

- La función `seleccion_ruleta` toma la población y las aptitudes correspondientes a cada individuo como entrada.
- Luego calcula la probabilidad acumulada de selección para cada individuo y utiliza números aleatorios para seleccionar individuos de acuerdo con esta probabilidad acumulada.
- La función retorna la lista de individuos seleccionados.



RULETA

```
import random

def seleccion_ruleta(poblacion, aptitudes):
    """ Selecciona individuos de la población utilizando el operador de ruleta. Parámetros: - poblacion
    (list): Lista de individuos en la población. - aptitudes (list): Lista de aptitudes correspondientes a
    cada individuo. Retorna: - list: Lista de individuos seleccionados mediante el operador de ruleta. """
    total_aptitudes = sum(aptitudes)

    probabilidad_acumulada = [sum(aptitudes[:i+1]) / total_aptitudes for i in range(len(aptitudes))]
    seleccionados = []
```

RULETA

```
# Seleccionar individuos usando la ruleta

for _ in range(len(poblacion)):
    r = random.random()
    for i, prob in enumerate(probabilidad_acumulada):
        if r <= prob:
            seleccionados.append(poblacion[i])
            break
    return seleccionados
```

RULETA

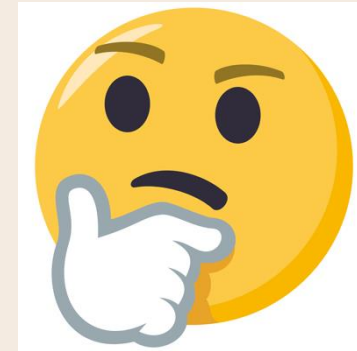
```
def main():  
    # Ejemplo de uso  
  
    poblacion = ["Individuo1", "Individuo2", "Individuo3", "Individuo4"]  
    aptitudes = [0.8, 0.6, 0.5, 0.7]  
  
    # Seleccionar individuos utilizando el operador de ruleta  
    seleccionados = seleccion_ruleta(poblacion, aptitudes)  
  
    # Mostrar los individuos seleccionados  
    print("Individuos seleccionados:", seleccionados)  
  
if __name__ == "__main__":  
    main()
```


HICE TRES EJECUCIONES

- aptitudes = [0.8, 0.6, 0.5, 0.7]
- ['Individuo1', 'Individuo1', 'Individuo4', 'Individuo4']



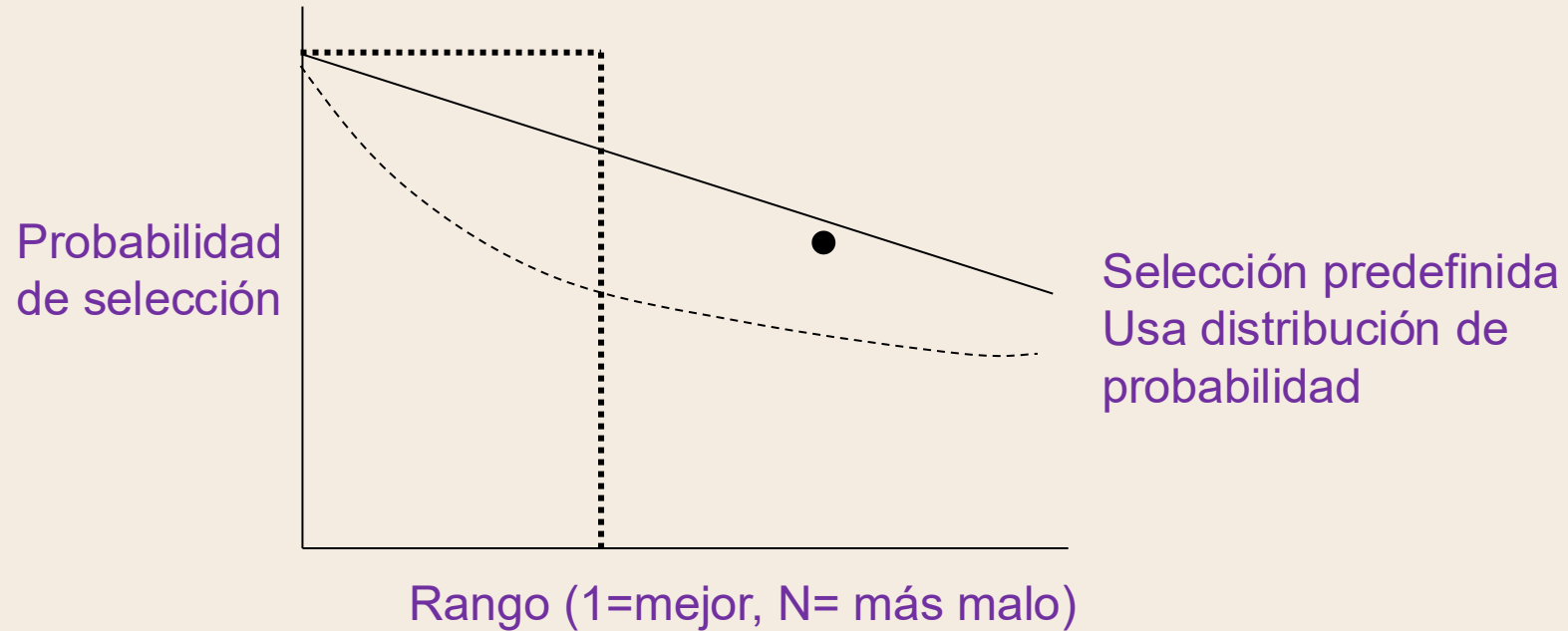
- aptitudes = [0.8, 0.6, 0.5, 0.7]
- ['Individuo1', 'Individuo4', 'Individuo3', 'Individuo1']



- aptitudes = [0.8, 0.6, 0.5, 0.7]
- ['Individuo3', 'Individuo1', 'Individuo3', 'Individuo3']



SELECCIÓN BASADA EN RANKING



Se hace un ranking de la población de acuerdo al fitness

Elitismo -> el mejor es elegido con probabilidad 1

- 1. Evaluación de la aptitud:** Se evalúa la aptitud de cada individuo en la población utilizando la función objetivo del problema que se está resolviendo. La función objetivo cuantifica qué tan bueno es un individuo en términos de la solución al problema.
- 2. Asignación de rangos:** Los individuos se clasifican en función de sus aptitudes, asignándoles un rango en orden descendente (el mejor individuo recibe el rango más alto). Por lo tanto, el individuo con la mejor aptitud tiene el rango más alto, el segundo mejor tiene el segundo rango, y así sucesivamente.
- 3. Probabilidad de selección:** Se asigna una probabilidad de selección a cada individuo con base en su rango.
- 4. Selección:** Se seleccionan individuos para la reproducción utilizando estas probabilidades. Los individuos con rangos más altos tienen una mayor probabilidad de ser seleccionados, pero aún existe la posibilidad de que individuos con rangos más bajos también sean elegidos.

- La idea detrás de la selección basada en ranking es promover una mayor diversidad genética en la población y evitar que la selección se concentre demasiado en los individuos con las mejores aptitudes. Esto puede ayudar a mantener la exploración del espacio de búsqueda y prevenir la convergencia prematura hacia soluciones subóptimas.

SELECCIÓN POR TORNEO

- ❖ Se elige 2 miembros de la población, al azar, Padre1 = el mejor de ellos.
- ❖ Se elige 2 miembros de la población, al azar, Padre2 = el mejor de ellos.



ALGORITMOS DE ESTADO ESTACIONARIO

- ❖ Un miembro de la población cambia en el tiempo t , en vez de la generación completa
- ❖ Población inicializada al azar
- ❖ Se hace ranking (orden) de la población por *fitness*
- ❖ * Se elige pares de padres usando selección basada en ranking
- ❖ Reproducción para generar descendientes
- ❖ Agregar descendientes en la posición correcta (ordenados) en la población (sin repeticiones)
- ❖ Los últimos del ranking *se van al infierno...*
- ❖ Ir a *, a menos que se cumpla el criterio de detención

ASPECTOS A TENER EN CUENTA

- Elección de cuestiones básicas de implementación:
 - Representación
 - Tamaño de la población, porcentaje de mutación, ...
 - Políticas de selección y eliminación
 - crossover, operadores de mutación
- Criterio de término
- Rendimiento, escalabilidad
- Una solución es sólo tan buena como la función de evaluación (la parte más difícil)

BENEFICIOS DE LOS ALGORITMOS GENETICOS

- Conceptos centrales son fáciles de entender.
- Modulares, se pueden separar de la aplicación.
- Permiten resolver problemas de optimización multi-objetivo.
- Buenos para ambientes “ruidosos”.
- Siempre entregan una respuesta; la que habitualmente mejora en el tiempo.
- Inherentemente paralelos; se pueden trabajar de forma distribuida.

BENEFICIOS DE LOS ALGORITMOS GENÉTICOS (CONT.)

- Se pueden mejorar las aplicaciones basadas en AGs a medida que se incrementa la experiencia en el dominio del problema.
- Es fácil reutilizar aplicaciones previas (o alternativas).
- Es posible construir bloques básicos para aplicaciones híbridas.
- Existe bastante historia acumulada sobre sus potenciales usos y aplicaciones.

CUANDO USAR UN AG

- Cuando las soluciones alternativas son muy lentas o muy complicadas
- Se requiere una herramienta exploratoria para analizar casos nuevos
- Cuando existen problemas similares que ya han sido resueltos usando AGs
- Para hibridizar; es decir para combinar con una solución existente que usa otras técnicas
- Cuando los beneficios de los AGs encajan bien con el problema

TIPOS DE APLICACIONES TÍPICAS CON AG

DOMINIO	APLICACIONES
Control	Evasión de misiles, balance de carga, gasoductos...
Diseño	Layout de circuitos, diseño de aeronaves, redes de comunicación
Scheduling	Industria manufactura, asignación de recursos
Robótica	Planificación de trayectorias
Machine learning	Diseño de redes neuronales, sistemas de clasificación
Procesamiento de señales	Diseño de filtros
Juegos	Póker, damas, ajedrez, dilema del prisionero
Optimización combinatoria	Set covering, diseño de rutas, coloración de grafos

CONVERSEMOS UN POCO DE GRAFOS

- Para darle una mirada...

GRAFOS

- La teoría de grafos es una rama de las matemáticas discretas y de las matemáticas aplicadas, que usa diferentes conceptos de diversas áreas como
 - combinatoria
 - álgebra
 - probabilidad
 - geometría de polígonos
 - aritmética y
 - topología
- Actualmente ha tenido mayor preponderancia en el campo de la informática,
 - Ciencias de la computación
 - Telecomunicaciones
 - Aplicaciones de IA

ACTUALIDAD

Graph-based Artificial Intelligence (AI) es un término usado para describir metodologías en las cuales los grafos juegan un papel central en la representación y mejoramiento de sistemas inteligentes.

- Information Sciences (Elsevier)
 - Graph-based solutions for Artificial Intelligence
 - Special Issue. A publicar en noviembre 2026

MOTIVACIÓN

- Los grafos, como estructuras matemáticas compuestas de nodos (vértices) y aristas (arcos) representan de forma natural relaciones, dependencias, e interacciones entre entidades.
- Esta propiedad intrínseca los hace adecuados para el modelamiento de datos con estructuras relacionales explícitas (por ejemplo, moléculas, redes sociales, o sistemas de transporte).

CONCEPTOS RELEVANTES EN IA

- **Knowledge Graphs (Grafos de conocimiento):** Mapea hechos del mundo real, entidades, y relaciones semánticas para fundamentar los argumentos de los LLMs y reducir las alucinaciones.
- **Graph Neural Networks (GNNs):** Modelos especializados de deep learning que procesan datos estructurados en la forma de grafos, a través de mensajes entre nodos conectados.
- **Planning Graphs:** Representaciones estructurales usadas en razonamiento automático y problem-solving para mapear secuencias de estados y acciones hacia un objetivo (meta).

CONCEPTOS RELEVANTES EN IA

- **GraphRAG (Retrieval-Augmented Generation):** Combina búsqueda vectorial con grafos de conocimiento para recuperar data en esquema multi-hop (obtención de datos y conclusiones desde diferentes contextos). En general, datos fuertemente relacionados para consultas IA.
- **Agentic Workflows (Graph Engineering):** Orquestación de sistemas multiagentes como nodos de redes donde las tareas y los datos realizan ciclos entre agentes especializados.

GRAFOS E IA

- La unión entre **grafos** (redes de nodos y conexiones) e **Inteligencia Artificial** permite que los modelos *comprendan* contextos complejos, reduzcan errores y procesen datos interconectados. Esta sinergia se aplica principalmente en redes neuronales especializadas, bases de datos estructuradas y sistemas de razonamiento autónomo

APLICACIONES EN IA

- **Descubrimiento de medicamentos:** A través del modelamiento de átomos como nodos y enlaces químicos como aristas. De esta forma se pueden predecir propiedades moleculares.
- **Sistemas de Recomendación:** Conectando usuarios a ítems a través de la interacción de aristas para sugerencias personalizadas (ejemplo: streaming y plataformas comerciales).
- **Fraude & Ciberseguridad:** Detección de comportamientos anómalos, movimientos laterales no autorizados, patrones de delitos financieros a través de procesos transaccionales.
- **Optimización:** Resolución de rutinas complejas, logística y desafíos en la administración de tráfico usando algoritmos como A*.

CONCEPTOS CLAVE

- **Redes Neuronales de Grafos (GNNs):** Modelos de aprendizaje profundo adaptados para procesar datos en forma de grafos en lugar de tablas, capturando relaciones locales y globales.
- **Grafos de Conocimiento (Knowledge Graphs):** Redes semánticas que estructuran hechos del mundo real (entidades y relaciones) para dar una base de datos lógica a los modelos.
- **IA Agéntica:** Sistemas autónomos que usan grafos como memoria estructural para planificar acciones basadas en datos verificables, en lugar de solo probabilidades.

GRAFOS Y LLM

- Los grafos de conocimiento (KG) son redes de información estructurada que conectan entidades reales (como personas, lugares o conceptos) mediante relaciones lógicas. Combinados con los grandes modelos de lenguaje (LLM), actúan como su "memoria de datos duros", reduciendo drásticamente las alucinaciones y aportando contexto preciso y verificable.
- **Cómo se complementan**
- **El LLM:** Aporta fluidez conversacional, capacidad de síntesis y entiende texto suelto o desestructurado.
- **El Grafo:** Aporta una base de hechos rígida, lógica y estructurada, evitando que el modelo invente datos.

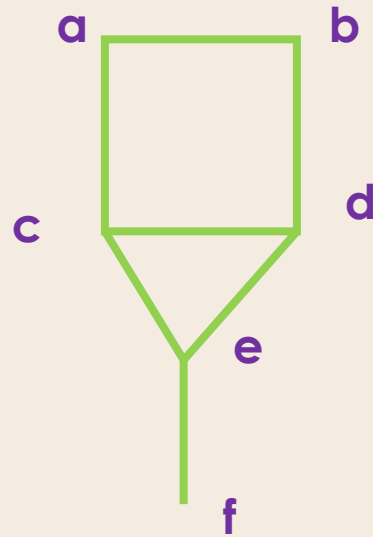
DICHO DE OTRA MANERA

- La combinación de grafos y grandes modelos de lenguaje (LLMs) crea una sinergia poderosa.
- Los grafos aportan memoria explícita y relaciones (nodos y aristas) mientras que los LLMs suministran comprensión del lenguaje natural, razonamiento y generación de texto.

DEFINICIONES BÁSICAS

- Un grafo G , consiste de $G = (V, E)$, donde
 - V es un conjunto de vértices (nodos, puntos)
 - E es un conjunto de pares no ordenados de vértices
- Los elementos de E se llaman aristas (líneas, links)
- En general las aplicaciones más conocidas usan grafos finitos (i.e., $|V|$ es finito).

EJEMPLO

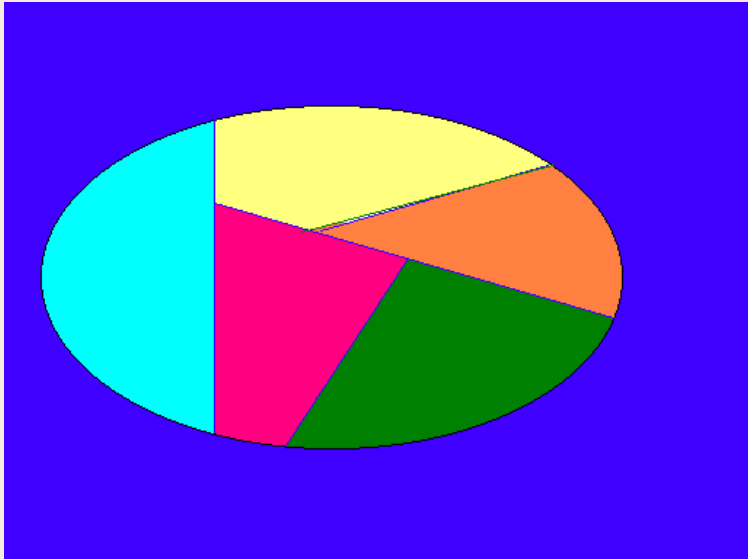


$$V = \{a, b, c, d, e, f\}$$

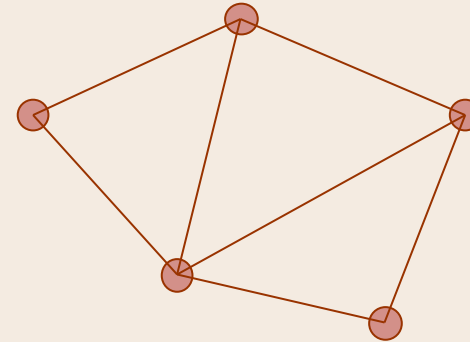
$$E = \{\{a, b\}, \{a, c\}, \{b, d\}, \{c, d\}, \{c, e\}, \{d, e\}, \{e, f\}\}$$

MAPAS VERSUS GRAFOS

Un mapa es un dibujo bidimensional con regiones a ser coloreadas.

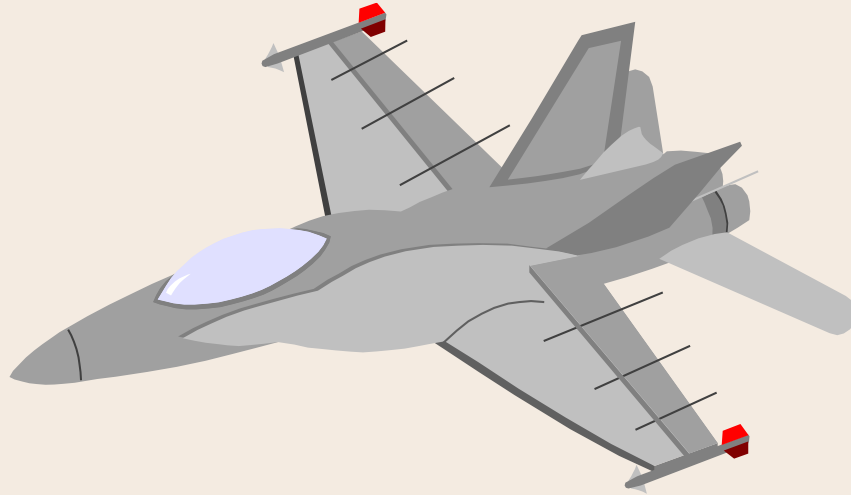


Un grafo es un objeto matemático construido de puntos conectados con líneas.



EL PROBLEMA DE LOS VUELOS

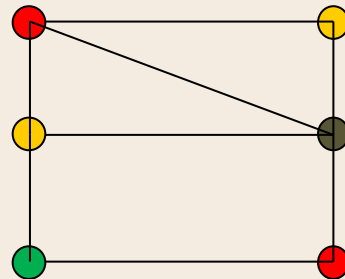
- Dado un conjunto de aviones que llegan en diferentes momentos, ¿cuántas mangas diferentes se necesitan para atenderlos a todos?



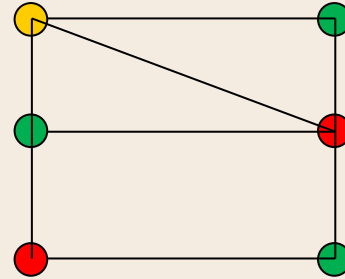
DEFINICIÓN

- Por coloración de vértices de un grafo G , se entiende la asignación de colores a los vértices de G , un color a cada vértice, de forma tal que vértices adyacentes tienen colores diferentes.

- Ejemplo:



○ también:



¿Cuál es el mínimo de colores necesarios?

OBS. Se considera solamente grafos simples, ya que si hay un loop no hay coloración posible.

COLORACIÓN EN GRAFOS

- Si $V(G)$ es un conjunto de cursos de la universidad, con aristas entre los cursos que tienen estudiantes en común, el número cromático del grafo es el mínimo de unidades de tiempo necesario para programar las clases sin que haya conflictos.

NO ES POR MOLESTAR PERO...

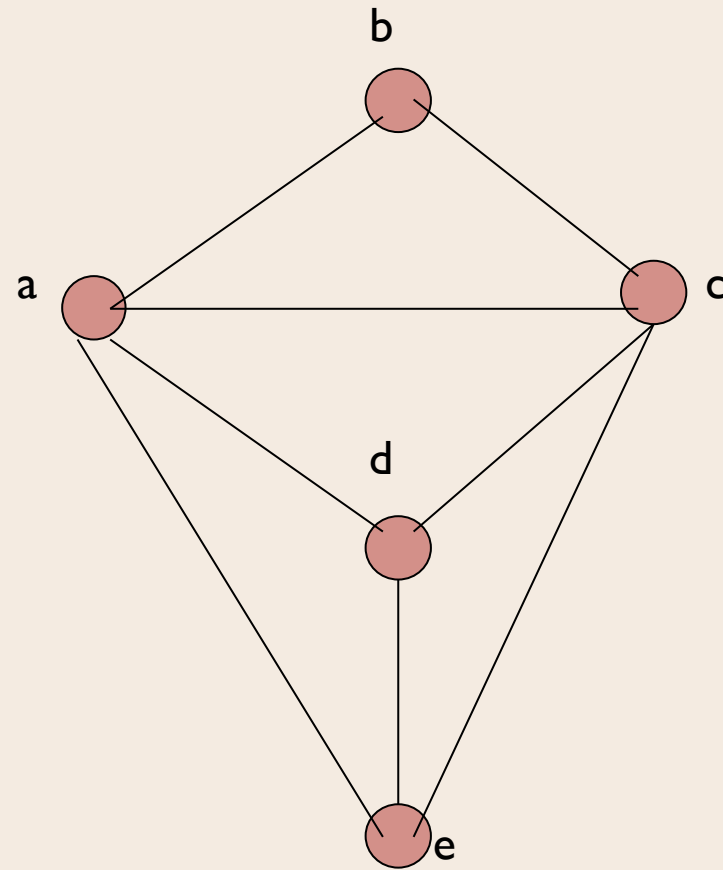
- En muchas instituciones (como el Congreso por ejemplo), es necesario realizar trabajo en múltiples comisiones.
- Podríamos optimizar los tiempos de trabajo de las comisiones para que, a los legisladores, les quede más espacio para las reuniones plenarias.
- Es sólo una idea.

DEFINICIÓN

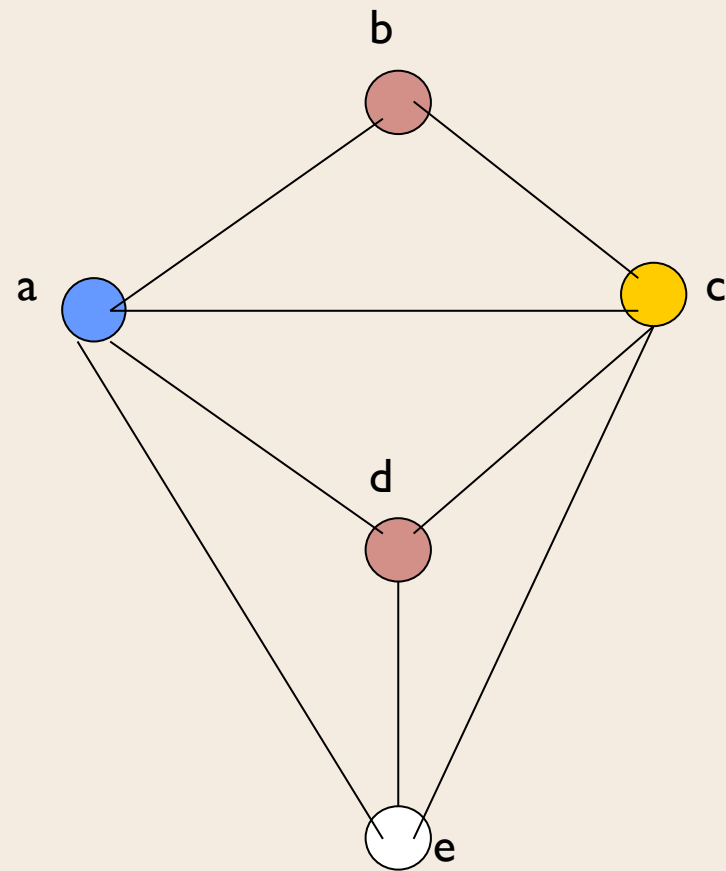
- Una k -coloración de G es un rotulado $f:V(G) \rightarrow \{1, \dots, k\}$.
- Un grafo G es k -coloreable si tiene la propiedad de una k -coloración. El número cromático $\chi(G)$ es el mínimo k tal que G es k -coloreable. Si $\chi(G) = k$, pero $\chi(H) < k$ para cada subgrafo propio H de G , entonces G es k -crítico.

EJEMPLO

$\aleph(G) = ?$



EJEMPLO



$$\aleph(G) = 4$$

PROXIMA SEMANA

- Swarm Intelligence
 - Colonias de hormigas